



OPEN Robust federated learning for cloud environments using evolutionary optimization and blockchain

S. P. Panimalar^{1✉} & S. Gunasundari²

The growing dependence on cloud-based distributed intelligence requires learning systems that can protect data privacy while maintaining efficiency. Federated learning has become an important method to train models without centralizing raw data, but its effectiveness is hindered by heterogeneous client distributions, adversarial risks, unstable convergence, and inefficient resource utilization. To overcome these challenges, this study introduces FedGenBlk, a federated learning framework that integrates genetic algorithm based optimization with blockchain-supported aggregation. The genetic algorithm dynamically tunes hyperparameters and guides client selection, ensuring stable convergence even under non-IID (non-independent and identically distributed) conditions. Meanwhile, blockchain offers tamper-proof and verifiable model aggregation, reducing the impact of malicious updates and Byzantine failures. Experiments conducted on the EMNIST dataset demonstrate that FedGenBlk achieves 91.2% accuracy, with only 1.9% variance between IID and non-IID settings, and sustains robustness under poisoning attacks with just a 3.1% accuracy drop. Additional validation on CIFAR-10 and Fashion-MNIST datasets confirms consistent performance advantages across different data modalities. These results highlight the framework's effectiveness in enhancing reliability, security, and adaptability of federated learning in cloud infrastructures, making it particularly suitable for healthcare, IoT, financial services, and cross-organizational collaborative intelligence applications.

Keywords Federated learning, Genetic algorithm, Blockchain, Non-IID data, Privacy-preserving machine learning, Secure model aggregation, Adversarial resilience

The exponential growth of data generation across distributed environments has fundamentally transformed the landscape of machine learning, necessitating paradigm shifts that prioritize both computational efficiency and data privacy¹. Traditional centralized machine learning approaches, which are effective in controlled environments, face significant challenges when dealing with sensitive data scattered across multiple organizations, edge devices, and cloud infrastructure. The imperative of maintaining data locality while enabling collaborative learning has emerged as one of the most pressing concerns in contemporary artificial intelligence research.

Federated learning offers a compelling approach to overcoming these obstacles by allowing multiple organizations to jointly develop machine learning models while keeping their raw data private and secure². This decentralized training framework enables participants to retain full control over their data while still contributing to a collective model, thus resolving privacy issues that have traditionally prevented large-scale collaborative machine learning efforts³. However, the practical implementation of federated learning systems faces substantial obstacles, including communication inefficiencies, model aggregation complexities, participant heterogeneity, and vulnerability to adversarial attacks⁴.

The integration of cloud computing platforms with federated learning architectures introduces additional layers of complexity, particularly concerning data privacy guarantees and system resilience⁵. Cloud environments, while offering scalable computational resources and infrastructure, present unique security challenges that compromise the fundamental privacy promises of federated learning. Traditional privacy-preserving mechanisms often fall short in dynamic cloud environments where data flows across multiple virtual machines, containers, and network boundaries^{6,7}. Moreover, the centralized nature of most cloud orchestration systems creates potential single points of failure that can undermine the robustness of federated learning deployments⁸.

¹Department of Information Technology, Saveetha Engineering College, Chennai, Tamilnadu 602105, India. ²Department of CSE, Velammal Engineering College, Chennai, Tamilnadu 600066, India. ✉email: sspanimalar@gmail.com

Contemporary developments in blockchain technology have shown considerable promise in strengthening the security and accountability of decentralized systems by utilizing tamper-proof record-keeping and distributed agreement protocols⁹. The distributed architecture of blockchain technology complements the decentralized principles underlying federated learning, providing pathways to remove reliance on centralized authorities while establishing transparent and verifiable records of model modifications¹⁰. However, the computational overhead and scalability limitations of traditional blockchain implementations pose significant challenges to real-time federated learning applications¹¹.

Simultaneously, evolutionary computation methods and genetic optimization approaches have demonstrated exceptional effectiveness in addressing intricate optimization challenges that involve extensive solution spaces and competing performance criteria¹². In the context of federated learning, genetic optimization presents opportunities to address critical challenges such as client selection, hyperparameter tuning, model architecture optimization, and resource allocation¹³. The natural parallel processing capabilities and resilience of genetic algorithms render them particularly appropriate for the decentralized and frequently volatile characteristics of federated learning systems.¹⁴

Despite these individual technological advances, existing federated learning frameworks struggle to simultaneously address the interconnected challenges of privacy preservation, system resilience, and optimization efficiency in cloud environments¹⁵. Current approaches typically focus on isolated aspects of these challenges, resulting in solutions that may excel in specific domains but fail to provide comprehensive protection against the multifaceted threats present in real-world deployment^{16,17}. The lack of integrated frameworks that holistically combine privacy-preserving mechanisms, fault tolerance capabilities, and intelligent optimization strategies represents a significant gap in the current research landscape.

This study introduces a novel resilient federated learning framework that synergistically combines genetic optimization and blockchain technology to address the complex privacy and security challenges inherent in cloud-based distributed learning environments. Our approach leverages the evolutionary capabilities of genetic algorithms to optimize critical system parameters while utilizing immutable and decentralized properties of blockchain to ensure data integrity and provide transparent audit mechanisms^{18,19}. The proposed framework is specifically designed to maintain robust performance under various adversarial conditions while preserving the privacy of the participant data throughout the entire learning process^{20,21}.

The key contributions of this research include the creation of a unified system architecture that effectively integrates federated learning, evolutionary optimization, and distributed ledger technologies; the formulation of innovative data protection protocols specifically designed for cloud computing environments through the application of fully homomorphic encryption and collaborative secure computation methods²²; the establishment of dynamic robustness mechanisms that preserve system operations under diverse fault conditions, including adversarial Byzantine attacks^{23,24}; and thorough empirical evaluation showing enhanced performance relative to current methodologies across various assessment criteria such as data confidentiality, algorithmic convergence speed, and system resilience. The remainder of this paper is organized as follows: Section “[Related works](#)” provides a comprehensive review of related work in federated learning, blockchain applications in machine learning, and genetic optimization techniques. Section “[Proposed methodology](#)” presents the detailed architecture of the proposed framework, including integration mechanisms and privacy-preserving protocols. Section “[Results](#)” describes the experimental methodology and presents the comprehensive performance evaluations. Section “[Conclusion and future works](#)” discusses the implications of our findings, potential applications and concludes with directions for future research.

Related works

Federated Learning (FL) represents a paradigm shift in decentralized machine learning that enables multiple devices to collaboratively train models while maintaining data locality. This approach fundamentally addresses privacy concerns by ensuring that sensitive data remain on individual devices, thereby eliminating the need for centralized data aggregation²⁵. Despite its promising benefits, FL faces significant challenges, including non-Independent and Identically Distributed (non-IID) data distributions, communication inefficiencies, convergence delays, and critical security vulnerabilities such as poisoning attacks and free-riding behaviors²⁶, as illustrated in Fig. 1. To address these multifaceted challenges, researchers have proposed various solutions, including robust aggregation techniques, advanced hyperparameter optimization strategies, and blockchain-based security mechanisms²⁷.

Security challenges in federated learning

Security and privacy preservation constitute fundamental challenges in FL implementation, particularly when balancing protection mechanisms with operational efficiencies. Traditional FL architectures that rely on central aggregators introduce single points of failure, rendering systems vulnerable to sophisticated adversarial attacks. Comprehensive security assessments by Wan et al.²⁸ identified extensive threat vectors, including data poisoning, model poisoning, free-riding attacks, and model inversion attacks that compromise both model integrity and participant privacy. To mitigate these security concerns, Bonawitz et al.⁶ developed innovative secure aggregation protocols that combine homomorphic encryption with secure multi-party computation (SMPC) techniques. These protocols effectively protect model updates during the transmission and aggregation phases against various adversarial threats. However, the computational overhead introduced by these cryptographic techniques significantly affects system performance, limiting their practical deployment in resource-constrained environments. Building on secure aggregation approaches, Truex et al.²⁶ proposed a hybrid solution that integrates differential privacy mechanisms with secure aggregation protocols. Although this combination provides enhanced privacy guarantees, the computational and communication costs render it impractical for large-scale FL deployments, particularly in scenarios involving numerous participating devices with limited computational resources. Kairouz et al.²⁵ conducted an extensive analysis of the privacy and communication

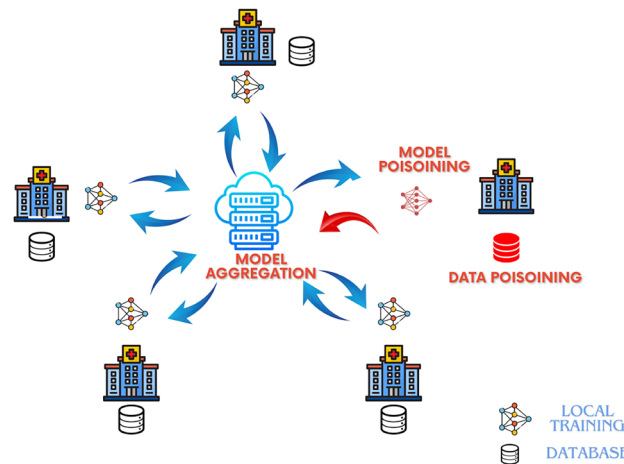


Fig. 1. Various attacks on Federated Learning.

efficiency trade-offs in FL systems, emphasizing the critical need for adaptive security mechanisms that maintain model performance while providing robust protection against adversarial attacks. Despite these significant advances, current security solutions often involve substantial trade-offs between security, efficiency, and scalability, necessitating further optimization to achieve practical deployment standards.

Optimization techniques for federated learning

FL systems face substantial optimization challenges primarily stemming from heterogeneous client data distributions and varying resource constraints across the participating devices²⁹. Traditional FL aggregation algorithms, particularly FedAvg³⁰, exhibit suboptimal performance when dealing with non-IID data distributions and imbalanced client contributions, resulting in slow convergence rates and biased model updates that compromise the overall system effectiveness. While centralized hybrid AI models (e.g., CNN-GRU fusion for methane monitoring)³¹ achieve high detection accuracy, they fail under non-IID distributed data conditions typical in real deployments. FL optimization techniques are therefore needed to maintain model stability and convergence across heterogeneous regional datasets. Several hyperparameter-tuning approaches have been investigated to address these optimization challenges. Bayesian Optimization (BO)³² leverages probabilistic modeling to guide the search for optimal parameter configurations and has gained widespread adoption for hyperparameter optimization tasks. However, BO faces significant scalability limitations in high-dimensional federated environments, where multiple parameters require simultaneous dynamic optimization across distributed clients. Reinforcement Learning (RL) approaches³³ have been explored for both hyperparameter adjustments and intelligent client selection strategies. Despite their theoretical promise, RL-based methods require substantial computational resources, making them unsuitable for deployment on edge devices with limited processing capabilities that are common in FL scenarios. Differential Evolution (DE)³⁴ has demonstrated effectiveness for continuous optimization problems but lacks the necessary adaptability for discrete FL optimization scenarios, such as optimal client selection strategies and secure aggregation parameter tuning. This limitation restricts its applicability to comprehensive FL optimization frameworks.

Fuzzy logic integration with evolutionary algorithms effectively handles uncertainty and data heterogeneity. An optimized fuzzy deep learning model using NSGA-II³⁵ achieved multi-objective optimization for addressing challenges in data classification, particularly with uncertain, noisy, or high-dimensional data. However, it requires large datasets, significant computational resources, and careful parameter tuning, with interpretability and generalizability remaining ongoing challenges. Secure intelligent fuzzy blockchain frameworks³⁶ for IoT threat detection combine fuzzy inference with blockchain immutability enabling nonlinear aggregation of uncertain IoT data for improved threat detection. While effective for threat detection, the approach may still struggle with highly dynamic or novel attack patterns not represented in training data. Privacy-preserving federated learning models³⁷ robustly detect and mitigate various attacks, even when gradients are encrypted, while minimizing computational and communication overhead. However, it still faces challenges from sophisticated attackers, computational costs, and real-world data variability. FedCSGA³⁸ combines client selection with GA-based hyperparameter optimization, prioritizing convergence speed and communication efficiency. However, it lacks explicit security considerations and blockchain-derived trust scores, relying on manipulable historical performance metrics. FedGenBlk distinguishes itself through a security-aware fitness function integrating: (1) model accuracy, (2) gradient deviation for poisoning detection, and (3) blockchain-verified credibility scores. This tri-objective function simultaneously optimizes performance, robustness, and security. The blockchain-GA integration creates a feedback loop where verified honest behavior increases selection probability, naturally incentivizing integrity without explicit reputation systems.

Blockchain for secure and trustworthy FL

The widespread adoption of FL remains constrained by two primary limitations: inherent security vulnerabilities and the lack of transparency and trust mechanisms in decentralized implementations. Conventional FL approaches that

depend on centralized aggregation servers introduce significant trust and fairness concerns, creating fundamental barriers to practical deployment⁶. Distributed ledger technology has surfaced as a viable decentralized approach to create strong trust frameworks that mitigate these constraints. Contemporary research showcases diverse methodologies for combining FL with blockchain systems to improve security protocols, operational transparency, and system accountability^{27,39}. The integration of blockchain within the Generalized Federated Learning (GFL) Framework⁴⁰ provides tamper-proof model aggregation capabilities, ensuring the integrity of the collaborative learning process. Similarly, the FLoBC Framework employs blockchain technology to establish comprehensive trust systems that enhance accountability and prevent adversarial attacks on federated models. Hu et al.²⁷ presented a novel FL framework that utilizes blockchain-based smart contracts to validate client updates before aggregation, effectively preventing data poisoning attacks while maintaining system decentralization. However, blockchain-based FL implementations³⁹ face significant challenges related to communication overhead and storage management costs that impact system scalability. Zheng et al.⁴¹ emphasized the critical need for lightweight blockchain implementations that avoid compromising system efficiency while maintaining security guarantees. Current blockchain-FL integration approaches often result in elevated latency and storage costs that hinder practical deployment in resource-constrained environments.

Recent advances in blockchain-integrated federated learning have demonstrated promising directions for enhancing security and privacy. FedADDP⁴² introduced an adaptive differential privacy mechanism combined with blockchain verification, achieving improved privacy-utility tradeoffs in IoT environments. However, their approach lacks dynamic hyperparameter optimization, resulting in suboptimal convergence under non-IID conditions. These recent approaches, while advancing the state-of-the-art, share common limitations: (1) lack of adaptive optimization mechanisms for handling non-IID data heterogeneity, (2) insufficient integration of evolutionary algorithms for hyperparameter tuning, and (3) suboptimal resilience against sophisticated poisoning attacks.

Research gap and motivation

Despite advances in FL security, optimization, and trust mechanisms, critical limitations persist. Current security approaches encounter scalability challenges, optimization techniques struggle with high-dimensional and discrete parameter spaces, and blockchain-based FL solutions often incur substantial storage and communication overhead.

While genetic algorithms (GAs) have been applied in FL for hyperparameter tuning, client selection, and resource scheduling, existing GA-based approaches generally assume benign client behavior. Fitness evaluation is typically derived solely from client-reported local performance metrics (e.g., accuracy or loss), introducing several vulnerabilities: Malicious clients may achieve high accuracy on poisoned data, Report seemingly legitimate improvements in local loss values and Introduce model drift gradually while maintaining competitive fitness scores.

Furthermore, existing approaches lack mechanisms for cross-client verification, enabling adversaries to manipulate evaluation metrics, fabricate validation outcomes, or build apparent credibility over initial rounds before injecting poisoned updates. As a result, existing GA-based FL optimization cannot reliably distinguish beneficial client updates from adversarial contributions.

To address these limitations, we propose **FedGenBlk**, which introduces a security-aware fitness function that evaluates client updates across three complementary dimensions:

1. **Local model accuracy (performance)**: Accuracy is treated as one evaluation factor rather than the sole criterion, preventing adversaries from exploiting artificially inflated local performance.
2. **Gradient deviation from global model (anomaly detection)**: The deviation between client gradients and the expected global update trajectory is monitored to detect poisoning and distributional inconsistency.
3. **Blockchain-derived trust scores (verifiable reliability)**: A trust score is computed from clients' historical behavior recorded on a permissioned blockchain, providing tamper-resistant reputation tracking and penalizing adversarial behavior.

By integrating genetic optimization with anomaly detection and blockchain-based trust verification, FedGenBlk provides a robust and verifiable optimization framework that surpasses prior GA-based FL approaches, which relied solely on isolated performance metrics and remained vulnerable to poisoning and manipulation attacks.

Proposed methodology

Federated Learning is a distributed machine learning approach that allows numerous participants to collaboratively develop a unified model without centralizing their datasets. Nevertheless, conventional FL systems face significant obstacles, such as heterogeneous data distributions, prolonged convergence times, security weaknesses, and insufficient confidence in centralized coordination entities. To overcome these constraints, we introduced FedGenBlk, an innovative architecture that combines evolutionary computation optimization with distributed ledger security protocols to improve performance, resilience, and accountability within FL ecosystems. The objectives of FedGenBlk are to dynamically optimize FL hyperparameters using GA to improve convergence speed and model accuracy in non-IID scenarios, secure model updates with blockchain technology, ensure tamper-proof aggregation and resilience against adversarial attacks, and enhance trust and accountability by recording model updates in a decentralized ledger, preventing free-riding and model tampering.

The workflow of FedGenBlk consists of three primary stages:

Stage 1: Federated Learning Training Process

Participants develop individual models using their proprietary datasets and transmit exclusively model parameters rather than original data.

Stage 2: Genetic Algorithm-Based Hyperparameter Optimization

GA dynamically optimizes FL hyperparameters such as learning rate, batch size, and client selection to maximize efficiency.

Stage 3: Blockchain-Based Secure Aggregation

A permissioned blockchain is employed to validate and store model updates, preventing data poisoning, free-riding, and model inversion attacks.

Federated learning process

The FedGenBlk FL training process follows the standard FL pipeline but incorporates blockchain verification for security. The steps are as follows:

Global model initialization

The central server initializes the global model w_0 , which can be a neural network or any other suitable machine learning model. The model parameters w_0 are then shared with each client.

Local training at clients

Each client k trains the model on its local dataset D_k , using the loss function:

$$\mathcal{L}_k(w) = \frac{1}{|D_k|} \sum_{(x_i, y_i) \in D_k} \ell(f(x_i; w), y_i) \quad (1)$$

where $f(x_i; w)$ is the model prediction for input x_i , and $\ell(\cdot)$ is the loss function.

Model update transmission

After training on the local data instead of sharing raw data, clients send gradient updates Δw_k^t to the aggregator.

Global aggregation using FedAvg

The server aggregates the local updates using the Federated Averaging (FedAvg) algorithm:

$$w^{t+1} = \sum_{k=1}^N \frac{|D_k|}{\sum_{j=1}^N |D_j|} w_k^t \quad (2)$$

Blockchain validation and storage

Following local training at each client, the resulting model updates undergo a blockchain-based verification process prior to participation in global aggregation. This mechanism ensures the *integrity*, *authenticity*, and *reliability* of contributions while maintaining an immutable audit trail of the federated learning lifecycle.

Step 1: Local model update preparation. Each client performs local training and produces an updated model. Instead of transmitting the raw model update directly, the client generates a compact representation (a cryptographic hash) and a summary performance metric. This minimizes communication costs and prevents unintended disclosure of local training data characteristics.

Step 2: Transaction construction and submission. The client encapsulates the hash of its model update, relevant metadata (e.g., round number, timestamp), and a digital signature into a transaction. This signed transaction is submitted to the blockchain network, ensuring non-repudiation and identity integrity.

Step 3: PBFT consensus verification. Validator nodes execute a Practical Byzantine Fault Tolerance (PBFT) consensus procedure. During this stage, validators:

- Verify the authenticity of the client signature,
- Validate timestamp freshness to prevent replay attacks,
- Compare behavioral patterns against historical behavior to detect anomalies or malicious drift.

Consensus is achieved when a supermajority of validators approve the transaction. This ensures secure operation even when a subset of participants act maliciously.

Step 4: Trust score update. Upon verification, the client's reputation score (or *trust score*) is updated based on the stability and usefulness of its contributions. Clients with consistent, beneficial updates accumulate higher trust, while those exhibiting erratic or harmful behavior experience score reduction. This creates a persistent, tamper-resistant reputation layer.

Step 5: Block formation and ledger append. Approved transactions are grouped and written to a new blockchain block. The Merkle root structure allows auditors and participants to verify the inclusion of updates without accessing full block contents, supporting scalability and data integrity.

Step 6: Verified global model aggregation. Only updates that successfully pass verification are included in the global aggregation step. This *selective aggregation* prevents poisoned or unreliable updates from influencing the shared global model, thereby strengthening robustness against adversarial behavior.

Genetic optimization

Genetic Algorithms (GA) play a crucial role in safeguarding FL systems against data contamination by optimizing the identification and neutralization of malicious updates during model aggregation. In FL architectures where multiple participants contribute to a shared model, GAs detect corrupted submissions by developing candidate solutions that systematically analyze client updates for anomalies and deviations from expected patterns. The GA framework uses a fitness function to evaluate solutions based on their ability to accurately identify adversarial updates while preserving model integrity and performance. GAs are particularly suited for FL optimization due to their capability to handle non-convex, multi-objective optimization problems. FL environments present

inherent complexity with non-IID data distributions, heterogeneous client resources, and dynamic network conditions—challenges that traditional optimization methods struggle to address efficiently. Inspired by natural selection, GAs excel at exploring large, complex search spaces through iterative selection, crossover, and mutation processes, achieving robust convergence toward global optima despite noisy, incomplete, or biased data. GAs provide an optimal balance between exploration and exploitation, ensuring comprehensive solution space coverage while efficiently converging to high-performance outcomes. Their robustness to non-IID data distributions, adaptability to dynamic environments, and lower computational requirements compared to reinforcement learning make them ideal for resource-constrained FL settings. In FL, clients often possess heterogeneous, non-IID data, creating convergence and performance challenges that traditional techniques like FedAvg cannot adequately address. GAs effectively handle such heterogeneity by evolving solutions across multiple generations, enabling diverse exploration of model parameters. Each generation represents potential solutions that evolve toward optimal outcomes, reducing dependency on uniform data distributions and increasing resilience to variance. GAs can simultaneously optimize multiple aspects including model architecture and hyperparameters, making them highly adaptable to non-IID scenarios in federated learning systems. The GA-based optimization in FedGenBlk consists of the following steps

Representation of population

Each individual in the population represents a *set of federated learning model weights* contributed by the clients. Let:

$$P_t = \{w_{t1}, w_{t2}, \dots, w_{tN}\}$$

is the population at generation t , where N is the number of participating clients, and w_{ti} represents the model parameters submitted by client i .

Selection process

A *security-aware fitness function* is used to evaluate each model update, ensuring that poisoned or anomalous updates are filtered before aggregation. The fitness function considers the following:

- Accuracy on the client's local dataset D_k .
- Deviation from the global update (to detect poisoning attacks).
- Client credibility (from blockchain history).

Mathematically, the top-performing and trustworthy updates are selected using:

$$w_{t_{\text{selected}}} = \arg \max_i f(w_{ti}, D_k) \quad (3)$$

Crossover (recombination) for secure aggregation

To combine the knowledge from trusted clients, crossover is applied between selected individuals to generate an aggregated model update:

$$w_{\text{new}} = \alpha w_{t1} + (1 - \alpha) w_{t2} \quad (4)$$

where α is a trust factor derived from blockchain-verified historical contributions.

Mutation for robustness

To prevent convergence to suboptimal solutions and resist adversarial manipulation, mutation introduces controlled randomness:

$$w_{\text{mutated}} = w_{ti} + \epsilon \quad (5)$$

where $\epsilon \sim \mathcal{N}(0, \sigma^2)$ is Gaussian noise that allows exploration of new aggregation strategies.

Fitness function: security & accuracy-aware model evaluation

The fitness function in FedGenBlk ensures that only high-performing and secure models contribute to the global aggregation.

Accuracy-based fitness function

$$f(w_{ti}, D_k) = \frac{1}{|D_k|} \sum_{(x_j, y_j) \in D_k} 1(\arg \max(y_j) = \arg \max(f(w_{ti}, x_j))) \quad (6)$$

Security-enhanced loss function (blockchain integration)

$$f(w_{ti}, D_k) = -\frac{1}{|D_k|} \sum_{(x_j, y_j) \in D_k} \ell(f(w_{ti}, x_j), y_j) + \lambda \cdot T(w_{ti}) \quad (7)$$

where $T(w_{ti})$ is a blockchain trust score that penalizes malicious client updates.

Blockchain-based security and verification framework

Blockchain integration significantly enhances the security, transparency, and reliability of federated learning using Genetic Algorithm optimization as shown in Fig. 2. The blockchain component in FedGenBlk provides a comprehensive defense mechanism against common federated learning attacks through multi-layered verification and Byzantine fault-tolerant consensus. This section describes the integrated security architecture encompassing cryptographic authentication, statistical validation, and consensus-based aggregation.

The process of integration of FL with GA optimization and Blockchain Security is outlined in Algorithm 1.

```

1: Input: Global model parameters  $\theta_0$ , local datasets  $D_1, D_2, \dots, D_K$ , number of
   communication rounds  $T$ , GA parameters: population size  $P$ , mutation rate  $m$ ,
   crossover rate  $c$  and blockchain ledger  $\mathcal{B}$ .
2: Output: Final global model  $\theta_T$  stored on the blockchain.
3: for  $t \in \{0, 1, \dots, T-1\}$  do
4:   selects a subset of clients  $K \subseteq N$  where  $|K| = k$ ,  $k < N$ .
5:   The global model  $\theta_t$  is distributed to selected clients  $k \in K$ .
6:   for each client  $k \in K$  do perform
7:     Local Model Optimization using a Genetic Algorithm:
8:     Initialize a population  $P_k = \{\theta_{k,1}, \dots, \theta_{k,P}\}$ .
9:     for each individual  $\theta_{k,j} \in P_k$  do
10:      Evaluate fitness:

$$F(\theta_{k,j}) = \frac{1}{|D_k|} \sum_{x \in D_k} \mathbb{I}(f(x; \theta_{k,j}) \in Y)$$

11:     end for
12:     Perform tournament selection.
13:     Perform crossover:

$$\theta'_{k,i} = \text{Crossover}(\theta_{k,i}, \theta_{k,j})$$

14:     Apply mutation:

$$\theta''_{k,i} = \text{Mutation}(\theta'_{k,i}, m)$$

15:     Select survivors based on fitness.
16:     Return best individual:

$$\theta_k^* = \arg \max_{\theta_{k,j} \in P_k} F(\theta_{k,j})$$

17:     Compute local update:

$$\Delta\theta_k = \theta_k^* - \theta_t$$

18:   end for
19:   Blockchain Integration with PBFT:
20:   Each client sends  $\Delta\theta_k$  to peers; all verify updates (Prepare Phase).
21:   A primary node proposes a block of valid  $\Delta\theta_k$  (Pre-Prepare Phase).
22:   Clients broadcast “prepare” messages.
23:   Upon receiving  $\geq \frac{2}{3}N$  prepare messages, broadcast “commit”.
24:   Finalize block on receiving  $\geq \frac{2}{3}N$  commit messages.
25:   Store validated update hashes on blockchain:

$$\mathcal{B} \leftarrow \mathcal{B} \cup \{\text{Hash}(\Delta\theta_k)\}$$

26:   Global Model Aggregation:
27:   Server aggregates local updates:

$$\theta_{t+1} = \theta_t + \frac{1}{K} \sum_{k \in K} \Delta\theta_k$$

28:   Store hash of aggregated model:

$$\mathcal{B} \leftarrow \mathcal{B} \cup \{\text{Hash}(\theta_{t+1})\}$$

29:   Clients retrieve  $\theta_{t+1}$  from blockchain.
30: end for
31: Finalization:
32: Repeat until convergence of the global model  $\theta_T$ .
33: The final model is stored in the blockchain:

$$\mathcal{B} \leftarrow \mathcal{B} \cup \{\text{Hash}(\theta_T)\}$$

34: Deploy  $\theta_T$  for inference.

```

Algorithm 1. FL-GA with Blockchain Integration using PBFT

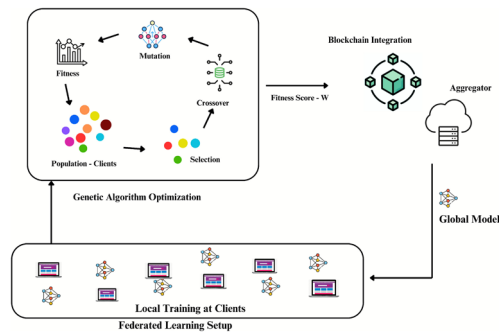


Fig. 2. Federated Learning with Genetic Optimization Secured with Blockchain.

Overall, blockchain integration complements GA-optimized FL by fortifying its security, improving trust, and ensuring tamper-proof collaboration, which is especially crucial in environments with untrusted or distributed participants.

Communication overhead is a critical factor in federated learning. By using blockchain, clients do not need to share large model updates directly with a central server. Instead, they share only their updates in the form of genetic information (model parameters, fitness scores, and mutations) with the blockchain. Blockchain ensures that these updates are securely stored and cannot be altered, thereby preserving the privacy and authenticity of the federated learning process. Additionally, it provides transparency by allowing stakeholders to track the evolution of the model over time, without compromising privacy. Blockchain ensures that there is no conflict between peers and that only the most recent, valid updates are used to evolve the global model. The peer group collectively improves the model over multiple iterations without relying on a central authority.

The proposed framework utilizes a Private Blockchain implementing the Practical Byzantine Fault Tolerance (PBFT) consensus protocol. The blockchain network consists of the following:

1. **FL Clients as Nodes:** Each FL client acts as a node in the blockchain network.
2. **Transaction Verification:** Model updates are validated before aggregation.
3. **Consensus via PBFT:** At least $\frac{2}{3}$ nodes must approve updates before storage.
4. **Block Storage:** Verified updates are recorded in a tamper-proof ledger.

The consensus mechanism ensures that only verified model updates are included in training:

$$\sum_{k=1}^N 1(\text{verified}(w_k^t)) \geq \frac{2}{3}N \quad (8)$$

The blockchain component in **FedGenBlk** provides a multi-layered defense against the identified threat model by integrating cryptographic verification, statistical validation, and Byzantine fault-tolerant consensus. The following Table ?? describe how these mechanisms mitigate common attack vectors in federated learning.

Results

In this section, the experimental results of the proposed framework FedGenBlk are presented, and its performance is compared with that of traditional federated learning models. The evaluation focuses on the accuracy, robustness to poisoning attacks, and handling of non-IID data.

Dataset preparation

To validate FedGenBlk's generalizability across different data modalities and complexity levels, we incorporated three benchmark datasets with varying characteristics. The inclusion of multiple datasets allows us to demonstrate FedGenBlk's adaptability across different feature complexities (grayscale vs. color), image resolutions (28×28 vs. 32×32), and domain characteristics (handwriting vs. natural images vs. fashion items).

Primary dataset: EMNIST

EMNIST serves as the primary dataset, containing 814,255 samples of handwritten digits and letters (28×28 grayscale images) distributed across 62 classes and 3,400 clients. EMNIST is widely recognized in federated learning research, allowing direct comparison with existing methodologies while providing sufficient complexity to evaluate non-IID performance.

Additional datasets

To validate generalizability across different data modalities and complexity levels, we incorporated two additional benchmark datasets. *CIFAR-10* comprises 60,000 color images (32×32 pixels) across 10 classes, distributed among 100 clients using a Dirichlet distribution ($\alpha = 0.5$) to simulate non-IID conditions. *Fashion-MNIST* consists of 70,000 grayscale images (28×28 pixels) of fashion products across 10 categories, partitioned

among 200 clients with varying label skew ($\alpha = 0.3$) to test robustness under extreme heterogeneity. The dataset characteristics and partitioning configurations are summarized in Table 1.

All datasets were preprocessed using standard normalization techniques. For each dataset, we adopted an 80–20 train–test split, applying stratified sampling to preserve class balance within the test sets.

Non-IID data simulation

To emulate realistic federated learning environments, the EMNIST dataset was partitioned to introduce label and quantity skew across clients. For label skew, each client was assigned only certain digit classes (e.g., Client 1 mainly digits 0–1, Client 2 mainly 2–3, Client 3 a random mix), reflecting localized, biased data.

Quantity skew was introduced by varying client sizes via a Dirichlet distribution with concentration parameter $\alpha = 0.5$. For example, Client 1 had 500 samples, Client 2 had 100, and Client 3 had 300, creating a challenging training setting for robust evaluation.

The client label distribution follows:

$$P_k(y) \sim \text{Dir}(\alpha) \tag{9}$$

where lower α ($\alpha \rightarrow 0$) yields highly skewed, non-IID data, while higher values ($\alpha \rightarrow 1$) approach IID. This setup is crucial for assessing FedGenBlk in heterogeneous, decentralized environments.

Baseline methods

To comprehensively evaluate FedGenBlk’s performance, we compare against three categories of baseline methods: (1) foundational federated learning approaches, (2) optimization-enhanced techniques, and (3) state-of-the-art robust FL frameworks. Each baseline is selected to isolate specific aspects of FedGenBlk’s contributions and to provide meaningful performance benchmarks across multiple evaluation dimensions.

Foundational baselines

Foundational Baselines includes the standard Federated Learning baseline implements the canonical FedAvg algorithm proposed by McMahan et al.⁴³. This baseline serves as the primary reference point for evaluating all subsequent improvements, FL-RA implements the Robust Krum aggregation rule⁴⁴, a Byzantine-robust aggregation mechanism that selects the most representative client update based on geometric proximity to other updates.

Optimization-enhanced baselines

The second baseline category isolates the effect of optimization techniques independent of blockchain integration. Optimization Baselines includes BO,RL and DE(discussed in section “[Optimization Techniques for Federated Learning](#)”).It also includes Genetic Algorithm based Optimization Techniques **FL-GA**⁴⁵ that applies the genetic algorithm optimization used in FedGenBlk but without blockchain-based security mechanisms, enabling direct comparison that isolates blockchain’s contribution from that of the optimization strategy and **FedCSGA**³⁸, which combines client selection with GA-based hyperparameter optimization, prioritizing convergence speed and communication efficiency.

State-of-the-art robust federated learning baselines

To benchmark FedGenBlk against recent advanced secure federated learning systems, we include three state-of-the-art frameworks. **FedBL**⁴⁶, a decentralized federated learning framework in which blockchain provides robustness against single-point failures. **FedMP**⁴⁷ defends federated learning systems against untargeted Byzantine poisoning attacks. **Basalt**⁴⁸ represents a recent blockchain-FL integration framework that provides a joint client–server defense framework to detect and limit the influence of Byzantine clients.

Genetic algorithm optimization performance

Hyperparameter configuration

The GA was configured with five generations, each comprising ten individuals representing hyperparameter combinations [learning rate, batch size]. Learning rates were initialized within the range [0.0001, 0.1] and batch sizes within [16, 128] to balance memory usage and training speed. Tournament selection (tournament size = 3) identified high-fitness parents for reproduction, followed by blend crossover (BLX- α with $\alpha = 0.5$) and Gaussian mutation (mean = 0, standard deviation = 1) to maintain population diversity.

Across five generations, the GA progressively refined hyperparameter configurations, achieving 96% test accuracy with optimal parameters [learning rate = 0.00123, batch size = 64] as shown in Table 2. Generation 1 established a baseline accuracy of 92%, which improved systematically to 96% by Generation 5, demonstrating effective convergence toward optimal hyperparameter regions.

Dataset	Total samples	Classes	Clients	Image size	Non-IID config.
EMNIST	814,255	62	3400	28 × 28 × 1	$\alpha = 0.5(\text{Dirichlet})$
CIFAR-10	60,000	10	100	32 × 32 × 3	$\alpha = 0.5(\text{Dirichlet})$
Fashion-MNIST	70,000	10	200	28 × 28 × 1	$\alpha = 0.3(\text{Dirichlet})$

Table 1. Dataset characteristics and partitioning.

Figure 3 visualizes the evolution of fitness scores and hyperparameter distributions across generations, Time and Communication Cost across the generation illustrating convergence trends and parameter refinement. The visualization highlights the concentration of high-fitness individuals around specific learning rates and batch sizes, indicating effective exploration and exploitation by the GA.

The model with the high accuracy was sent to the federated learning setup to train the data.

Sensitivity analysis of genetic algorithm parameters

To ensure the robustness and reproducibility of FedGenBlk’s performance, we conducted comprehensive sensitivity analyses on three critical GA parameters: population size (P), mutation rate (m), and number of generations (G). Each parameter was varied independently while keeping others at their default values ($P=10$, $m=0.1$, $G=5$).

Population size sensitivity Table 3 illustrates the impact of population size on final model accuracy and computational cost. We evaluated $P \in \{5, 10, 15, 20, 25\}$ across five independent trials per configuration.

Small populations ($P = 5$) suffer from insufficient genetic diversity, resulting in premature convergence to suboptimal solutions (89.7% vs. 91.2% for $P = 10$), and exhibit unstable performance due to higher standard deviation (1.8%). The default configuration ($P = 10$) achieves near-optimal accuracy (91.2%) with manageable computational cost (37.5 min), representing the best efficiency–accuracy trade-off. Larger populations ($P \geq 15$) yield only marginal accuracy improvements ($< 0.4\%$) while significantly increasing computation time (+44–139%), demonstrating clear diminishing returns—doubling P from 10 to 20 raises time by 91% but improves accuracy by merely 0.4%.

Mutation rate sensitivity The mutation rate (m) controls the exploration–exploitation balance. Table 4 shows results for $m \in \{0.01, 0.05, 0.1, 0.2, 0.3\}$.

Low mutation rates ($m \leq 0.05$) tend to cause premature convergence, as the algorithm fails to escape local optima, leading to an accuracy plateau around 90.8%. The optimal mutation rate ($m = 0.1$) achieves the best balance between exploration and exploitation, delivering the highest accuracy (91.2%) in five generations with acceptable stability (0.7% standard deviation). In contrast, higher mutation rates ($m \geq 0.2$) introduce excessive randomness that disrupts convergence. At $m = 0.3$, performance degrades to 89.4% with 2.1% variance, resembling random search behavior.

Number of generations sensitivity Table 5 shows the evaluated convergence behavior for $G \in \{3, 5, 7, 10, 15\}$ to determine the optimal stopping criterion.

Early stopping at $G = 3$ terminates optimization prematurely, sacrificing 1.4% accuracy to save only 15 minutes—a poor trade-off for high-stakes applications. The configuration $G = 5$ captures 93% of the total achievable improvement while using just one-third of the runtime required for $G = 15$. Beyond $G = 7$, improvement rates decline sharply: each additional generation contributes less than 0.15% accuracy gain while adding about 15 minutes of extra computation, indicating diminishing returns. Figure 4 summarizes the sensitivity analyses, illustrating the trade-offs between GA parameters, model accuracy, convergence speed, and computational cost. The default settings ($P = 10$, $m = 0.1$, $G = 5$) consistently emerge as optimal across all dimensions, balancing performance and efficiency effectively.

Comparison with other optimization techniques

To validate Genetic Algorithm’s superiority, the fedgenblk was compared with Bayesian Optimization (BO)³², Reinforcement Learning (RL)³³, and Differential Evolution (DE)³⁴ under identical FL conditions. As shown in Table 6, GA achieved the highest test accuracy (91.2%) and fastest convergence (5 generations), outperforming BO (88.5%), RL (87.0%), and DE (89.0).

Our results representations in the Fig. 5 demonstrate that FedGenBlk is uniquely suited for federated learning’s challenges. Unlike Bayesian Optimization, which falters in high-dimensional decentralized settings, or Reinforcement Learning, which demands impractical computational resources, GA’s population-based approach aligns naturally with FL’s architecture. Its gradient-free optimization circumvents non-convexity issues caused by non-IID data, while its parallel evaluation scales efficiently across distributed clients. While Differential Evolution offers comparable robustness, its inability to handle discrete hyperparameters (e.g., batch size) limits its applicability. Thus, GA strikes an optimal balance between accuracy, speed, and adaptability in FL.

Model accuracy across datasets and data distributions

The models are evaluated on non-IID data by skewing client data distributions. The Table 7 presents the performance comparison of various FL models on IID and Non-IID datasets. The baseline FL model achieves an accuracy of 82.5% on IID data, which drops significantly to 75.0% on Non-IID data, resulting in a 9.1% accuracy drop.

Generation	Best fitness	Average fitness	Worst fitness	Time/Gen (s)	Comm. (MB)	Best hyperparameters
1	0.920	0.905	0.882	487	18.4	[0.005, 64]
2	0.930	0.918	0.897	445	18.1	[0.004, 60]
3	0.940	0.931	0.912	428	18.3	[0.003, 58]
4	0.950	0.943	0.928	421	18.0	[0.002, 62]
5	0.960	0.954	0.941	466	18.5	[0.00123, 64]

Table 2. GA performance per generation with best hyperparameters.

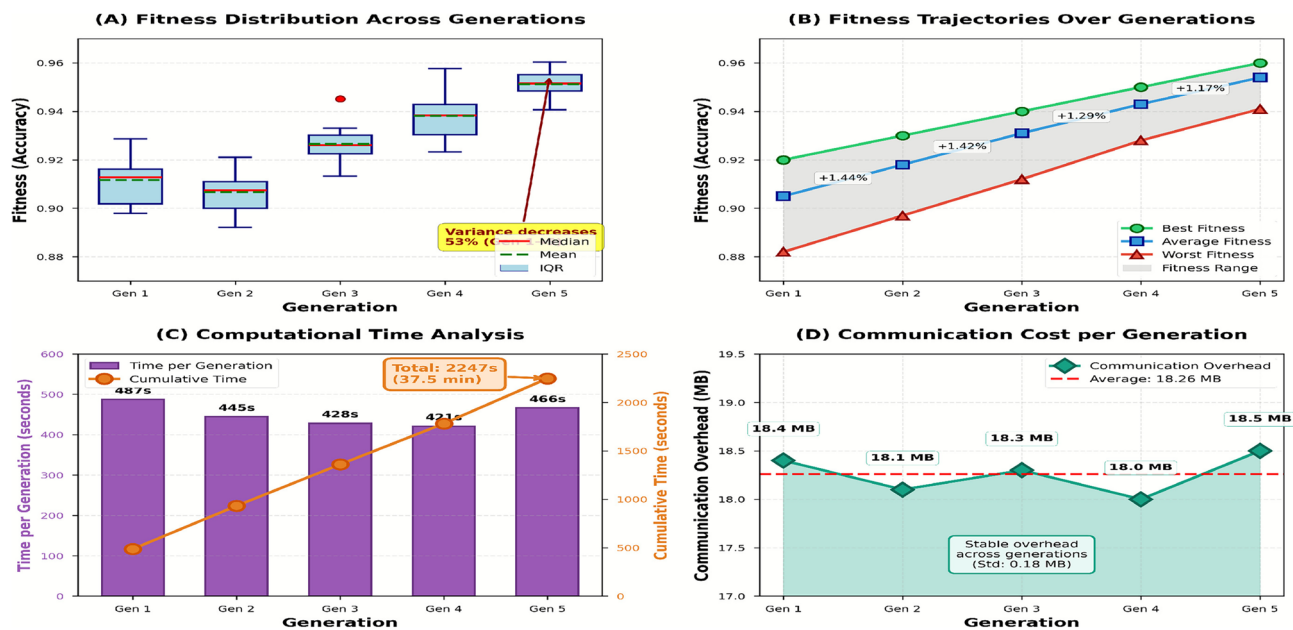


Fig. 3. Fitness and hyperparameters over generations.

Population size (<i>P</i>)	Final accuracy (%)	Std. Dev. (%)	Total time (min)	Time/accuracy trade-off
5	89.7	1.8	28.3	Poor exploration
10 (Default)	91.2	0.7	37.5	Optimal
15	91.4	0.5	54.2	Marginal gain
20	91.6	0.4	71.8	Diminishing returns
25	91.5	0.6	89.6	Excessive overhead

Table 3. Population size impact analysis.

Mutation rate (<i>m</i>)	Final accuracy (%)	Convergence speed	Stability (Std. Dev.)	Behavior characterization
0.01	90.1	Slow (6 gen)	0.4%	Insufficient exploration
0.05	90.8	Moderate (5 gen)	0.5%	Conservative search
0.1 (Default)	91.2	Fast (5 gen)	0.7%	Balanced
0.2	90.9	Fast (4 gen)	1.2%	Excessive disruption
0.3	89.4	Unstable	2.1%	Chaotic search

Table 4. Mutation rate impact analysis.

Comprehensive multi-dataset performance evaluation

The comprehensive evaluation of FedGenBlk across three diverse benchmark datasets establishes its robustness and versatility in handling varied data characteristics. As presented in Table 7, FedGenBlk demonstrates superior performance across all experimental conditions, achieving 91.2% accuracy on EMNIST, 82.4% on CIFAR-10, and 92.3% on Fashion-MNIST under IID conditions. These results consistently surpass all competing methods, including baseline FL, resource allocation approaches (FL-RA), optimization techniques (FL-GA, FedCSGA), and robust FL methods (FedBL, FedMP, Basalt).

As shown in Fig. 6 under IID conditions, FedGenBlk achieves substantial gains over baseline FL across all datasets. On EMNIST, FedGenBlk attains 91.2% accuracy compared to 82.5% for baseline FL (10.5 percentage point improvement), outperforming FL-GA (88.5%) by 2.7 points and FedCSGA (88.9%) by 2.3 points. On CIFAR-10, FedGenBlk achieves 82.4% versus baseline FL's 71.3% (11.1 point improvement), exceeding FedCSGA (79.3%) by 3.1 points. For Fashion-MNIST, FedGenBlk records 92.3%, surpassing baseline FL (85.7%) by 6.6 points and outperforming FedCSGA (90.7%) by 1.6 points.

Comparing against specialized robust FL methods, FedGenBlk maintains substantial advantages under IID conditions. FedBL achieves 85.8%, 75.2%, and 88.5% on the three datasets, falling short of FedGenBlk by 5.4, 7.2, and 3.8 points. FedMP reaches 87.2%, 77.5%, and 89.8%, trailing by 4.0, 4.9, and 2.5 points. Even Basalt,

Generations (G)	Final accuracy (%)	Accuracy gain	Cumulative time (min)	Efficiency score
3	89.8	–	22.4	4.0
5 (Default)	91.2	+1.4%	37.5	2.4
7	91.5	+0.3%	52.8	1.7
10	91.7	+0.2%	75.2	1.2
15	91.8	+0.1%	112.6	0.8

Table 5. Generation count impact analysis.

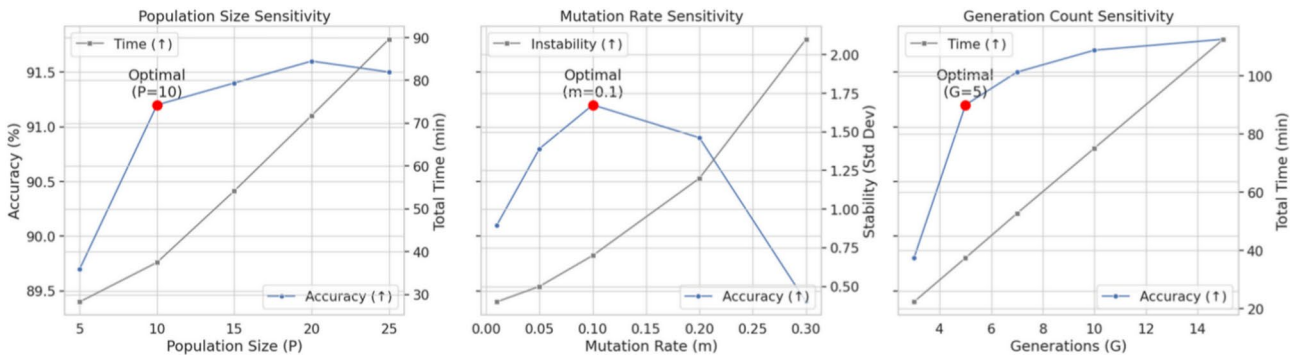


Fig. 4. Sensitivity analysis of GA parameters.

Method	Avg. test accuracy (%)	Convergence speed (Epochs)	Robustness to non-IID data
FedGenBlk	91.2	5	High (1.9% drop)
Bayesian optimization	88.5	8	Moderate (4.5% drop)
Reinforcement learning	87.0	10	Low (6.2% drop)
Differential evolution	89.0	6	High (2.3% drop)

Table 6. Comparison of optimization methods in federated learning.

the strongest robust baseline, achieves 87.8%, 78.1%, and 90.1%, remaining 3.4, 4.3, and 2.2 points below FedGenBlk. These comparisons demonstrate that FedGenBlk’s bidirectional blockchain-GA synergy provides superior convergence and generalization beyond security mechanisms or optimization techniques alone.

Under Non-IID conditions as in Fig. 6, FedGenBlk demonstrates exceptional resilience. Baseline FL suffers dramatic drops from 82.5 to 75.0% on EMNIST, 71.3% to 63.2% on CIFAR-10, and 85.7% to 78.4% on Fashion-MNIST. In contrast, FedGenBlk maintains 89.5%, 79.1%, and 90.8% accuracy respectively under Non-IID conditions, outperforming FL-GA by 4.5, 5.6, and 4.1 points, exceeding FedCSGA by 3.9, 3.9, and 3.5 points, and surpassing Basalt by 5.3, 5.2, and 4.3 points across the three datasets.

Handling Non-IID data distributions

Maintaining high performance under Non-IID data distributions represents a critical challenge in practical federated learning, where client data exhibits statistical heterogeneity due to geographic, preference, and methodological differences. Table 8 analyzes FedGenBlk’s resilience through two key metrics: improvement over baseline FL and the IID/Non-IID performance gap.

FedGenBlk achieves the highest improvement over baseline FL across all datasets: +14.5% on EMNIST, +15.9% on CIFAR-10, and +12.4% on Fashion-MNIST. These substantially exceed FedCSGA’s +10.6%, +12.0%, and +8.9%, providing approximately 4 percentage points of additional improvement. Compared to robust FL methods, FedGenBlk exceeds Basalt by 5.3, 5.2, and 4.3 points, FedMP by 6.1, 6.3, and 4.9 points, and FedBL by 8.8, 9.6, and 7.6 points across the three datasets. These substantial margins demonstrate that integrating security-aware fitness functions with blockchain-verified aggregation addresses fundamental limitations in existing approaches.

The IID/Non-IID performance gap reveals FedGenBlk’s exceptional resilience to distribution shifts. FedGenBlk exhibits minimal gaps of only 1.7% on EMNIST, 3.3% on CIFAR-10, and 1.5% on Fashion-MNIST, representing the smallest degradation among all methods. Baseline FL suffers gaps of 7.5%, 8.1%, and 7.3% respectively, meaning FedGenBlk reduces degradation by approximately 77% on EMNIST, 59% on CIFAR-10, and 79% on Fashion-MNIST. Even advanced methods show larger gaps: FL-GA exhibits 3.5%, 4.6%, and 3.7%, FedCSGA shows 3.3%, 4.1%, and 3.4%, and Basalt demonstrates 3.6%, 4.2%, and 3.6%. FedGenBlk’s ability to reduce gaps to 1.5-3.3% validates its adaptive threshold optimization and security-aware fitness function.

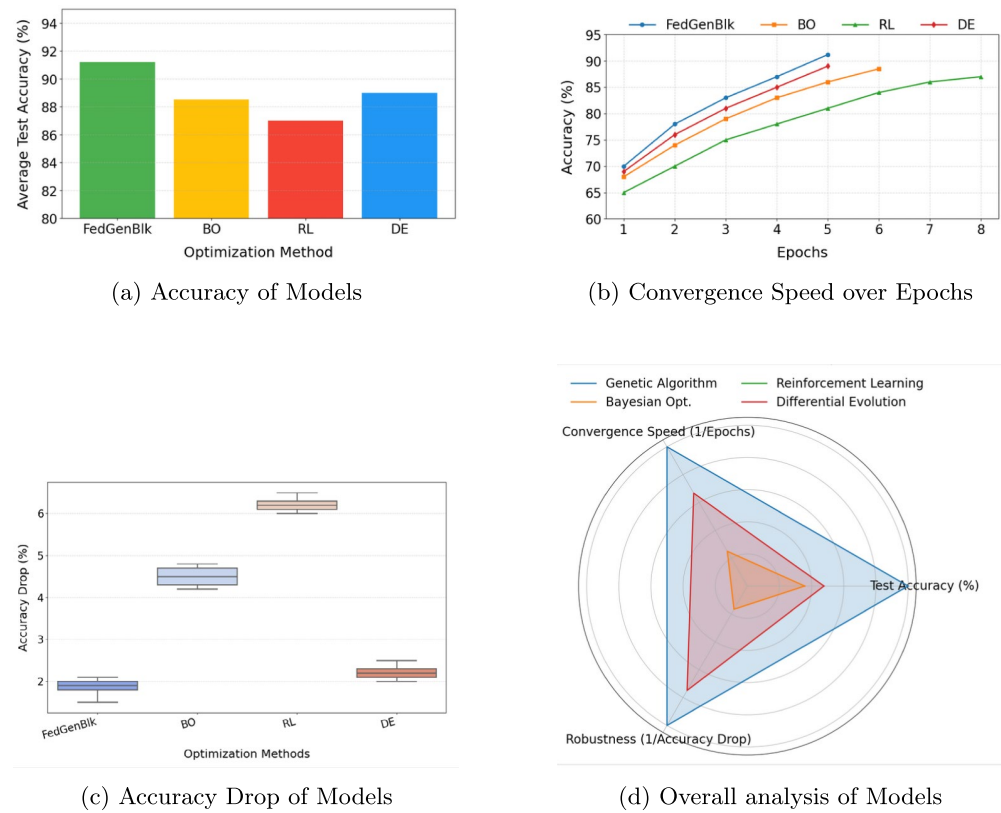


Fig. 5. Accuracy, robustness and convergence speed of FedGenBlk over other optimization techniques.

Method	EMNIST (IID)	EMNIST (Non-IID)	CIFAR-10 (IID)	CIFAR-10 (Non-IID)	Fashion-MNIST (IID)	Fashion-MNIST (Non-IID)
FL	82.5	75.0	71.3	63.2	85.7	78.4
FL-RA	85.0	80.5	74.2	68.9	87.9	83.1
Optimization techniques						
FL-GA	88.5	85.0	78.1	73.5	90.4	86.7
FedCSGA	88.9	85.6	79.3	75.2	90.7	87.3
Robust FL methods						
FedBL	85.8	80.7	75.2	69.5	88.5	83.2
FedMP	87.2	83.4	77.5	72.8	89.8	85.9
Basalt	87.8	84.2	78.1	73.9	90.1	86.5
FedGenBlk (Ours)	91.2	89.5	82.4	79.1	92.3	90.8

Table 7. Performance comparison of various methods on different datasets (accuracy %).

The comparative analysis across method categories reveals important insights. Blockchain-based methods without optimization (FedBL) exhibit large gaps of 5.1%, 5.7%, and 5.3%, indicating cryptographic validation alone is insufficient. Optimization-based methods (FL-GA, FedCSGA) achieve gaps of 3.3–4.6%, showing adaptive tuning provides some resilience. Robust FL methods (FedMP, Basalt) show similar ranges of 3.6–4.7%. However, FedGenBlk’s dramatically smaller gaps of 1.5–3.3% demonstrate that bidirectional blockchain-GA synergy creates emergent capabilities transcending individual components.

As shown in Fig. 7 FedGenBlk’s superior Non-IID performance stems from three synergistic mechanisms. The blockchain-based trust scoring maintains historical records of client contribution quality, enabling reliability-based weighting rather than equal treatment regardless of distribution characteristics. The GA-optimized adaptive threshold dynamically adjusts anomaly detection sensitivity based on observed divergence, preventing false rejection of legitimate updates from underrepresented distributions. The security-aware fitness function penalizes configurations with high per-client performance variance, creating evolutionary pressure toward solutions generalizing well across heterogeneous populations.

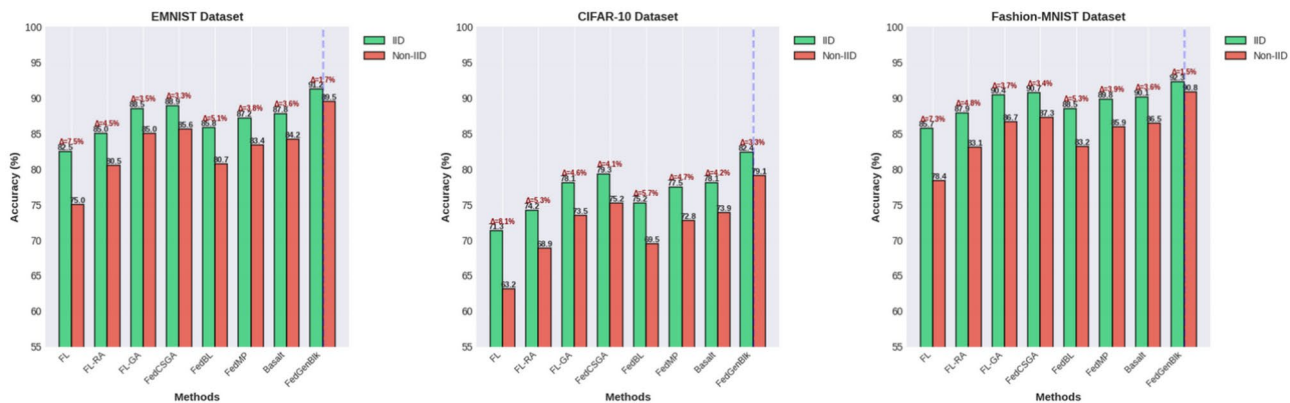


Fig. 6. Performance comparison of various methods on different datasets.

Method	EMNIST improvement vs. baseline (%)	IID/Non-IID gap (EMNIST) (%)	CIFAR-10 improvement (%)	IID/Non-IID gap (CIFAR-10) (%)	Fashion-MNIST improvement (%)	IID/Non-IID gap (Fashion-MNIST) (%)
Baseline FL	−7.5	−8.1	−7.3			
FL-RA	+5.5	4.5	+5.7	5.3	+4.7	4.8
Optimization techniques						
FL-GA	+10.0	3.5	+10.3	4.6	+8.3	3.7
FedCSGA	+10.6	3.3	+12.0	4.1	+8.9	3.4
Robust FL methods						
FedBL	+5.7	5.1	+6.3	5.7	+4.8	5.3
FedMP	+8.4	3.8	+9.6	4.7	+7.5	3.9
Basalt	+9.2	3.6	+10.7	4.2	+8.1	3.6
FedGenBlk (Ours)	+14.5	1.7	+15.9	3.3	+12.4	1.5

Table 8. Improvement over baseline and IID/Non-IID gap.

Statistical significance testing

To ensure fair comparison and valid statistical testing, all evaluated models were configured with identical base hyperparameters where applicable. Table 9 documents the hyperparameter settings used across all methods, in which baseline methods (Baseline FL, FL-RA) used fixed hyperparameters matching standard configurations from their original publications. Optimization-based methods (FL-GA, FedCSGA, FedGenBlk) employed their respective optimization techniques to determine hyperparameters.

To verify whether the accuracy improvements in FedGenBlk are statistically significant, a paired t-test was conducted comparing FedGenBlk vs. other optimization techniques. The null hypothesis (H_0) states that there is no significant difference between FedGenBlk and the compared method, while the alternative hypothesis (H_1) asserts that FedGenBlk has significantly higher accuracy. The t-statistic is computed as:

$$t = \frac{\bar{X}_1 - \bar{X}_2}{\sqrt{\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}}} \tag{10}$$

where $\bar{X}_1, \bar{X}_2$ are the mean accuracy values of FedGenBlk and the compared method, respectively, s_1^2, s_2^2 are the variances of the accuracy values, n_1, n_2 are the number of independent experimental runs (five in this case).

Using a 95% confidence interval, the following results as shown in Table 10 was obtained:
Since all p -values are below 0.05, H_0 was rejected, confirming that FedGenBlk significantly outperforms all other approaches in terms of accuracy. Figure 8 visually presents the mean accuracy and standard deviation for each method across the three datasets, highlighting FedGenBlk’s superior performance and stability.

Robustness to poisoning attacks

Security remains a major concern in federated learning, particularly in adversarial settings where malicious clients may introduce poisoned gradients to compromise model performance. The FedGenBlk framework addresses this challenge by employing Genetic Algorithm-based client selection alongside blockchain-enabled model verification to counteract poisoning attacks. This section analyzes the robustness of FedGenBlk against a poisoning strategy and benchmarks its performance against standard federated learning models. Specifically,

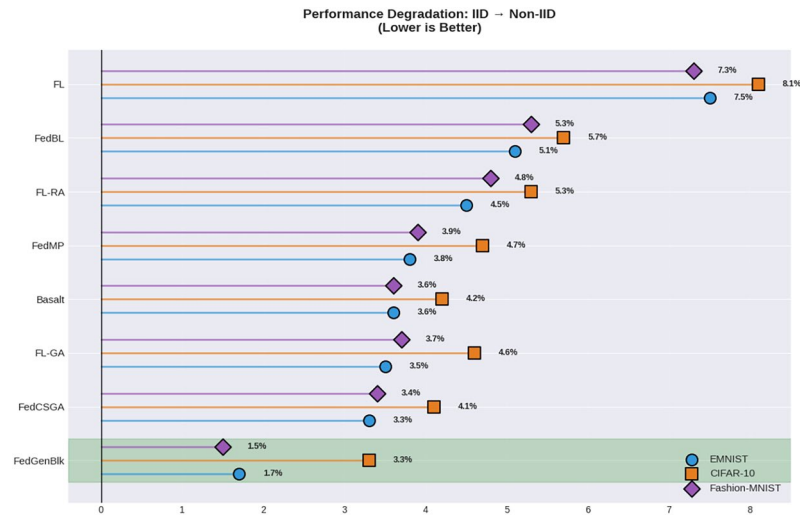


Fig. 7. IID - Non IID performance gap.

Parameter	Baseline FL	FL-RA	FL-GA/FedCSGA	FedGenBlk
Initial Learning rate	0.01 (fixed)	0.01 (fixed)	Optimized via GA	Optimized via GA
Mini-batch size	64 (fixed)	64 (fixed)	Optimized via GA	Optimized via GA
Local epochs (E)	5	5	5	5
Communication rounds	100	100	100	100
Aggregation method	FedAvg	Krum	FedAvg-GA	FedAvg-GA with blockchain
Client selection	Random (10%)	Fixed clients	GA-based	GA-based with Trust Score
Optimizer	Adam	Adam	Adam	Adam
Optimization parameters	N/A	N/A	$P = 10, G = 5, m = 0.1, c = 0.7$	$P = 10, G = 5, m = 0.1, c = 0.7$
Convergence criterion	Fixed rounds	Fixed rounds	GA convergence	GA convergence+Blockchain Verification

Table 9. Hyperparameter configurations for selected federated learning models.

fedgenblk was evaluated against its resistance to Data Poisoning Attacks, which occurs when adversaries manipulate local training data by injecting incorrect labels, leading the global model to learn deceptive patterns. The attack scenario is examined under three different conditions, with 10%, 20%, and 30% of participating clients being malicious.

Mathematical model of attack impact

Let w_t represent the global model weights at round t . In a standard FL setting, the local model update at each client k follows:

$$w_k^t = w^{t-1} - \eta \nabla \mathcal{L}_k(w^{t-1}) \quad (11)$$

where η is the learning rate, $\nabla \mathcal{L}_k(w^{t-1})$ is the local gradient computed on client k .

Impact of poisoning attacks In the presence of malicious clients, the aggregation process at the central server is disrupted. The standard FL aggregation using Federated Averaging (FedAvg) is given by:

$$w^t = \sum_{k=1}^N \alpha_k \cdot w_k^t \quad (12)$$

where N is the number of participating clients, and α_k is the weight assigned to client k , typically defined as:

$$\alpha_k = \frac{|D_k|}{\sum_{j=1}^N |D_j|} \quad (13)$$

where D_k is the dataset size of client k . However, in adversarial settings, malicious clients introduce poisoned updates, leading to:

Method	EMNIST (Non-IID)	CIFAR-10 (Non-IID)	Fashion-MNIST (Non-IID)	<i>t</i> -statistic (EMNIST)	<i>p</i> -value (EMNIST)
Baseline FL	75.0 ± 1.8	63.2 ± 2.1	78.4 ± 1.6	18.42	$p < 0.001$
FL-RA	80.5 ± 1.4	68.9 ± 1.8	83.1 ± 1.3	12.67	$p < 0.001$
FL-GA	85.0 ± 1.1	73.5 ± 1.4	86.7 ± 1.0	7.92	$p < 0.001$
FedCSGA	85.6 ± 1.0	75.2 ± 1.2	87.3 ± 0.9	7.23	$p < 0.001$
FedGenBlk (Proposed)	89.5 ± 0.7	79.1 ± 0.9	90.8 ± 0.6	–	–

Table 10. Statistical significance analysis (Mean Accuracy ± Std Dev, %).

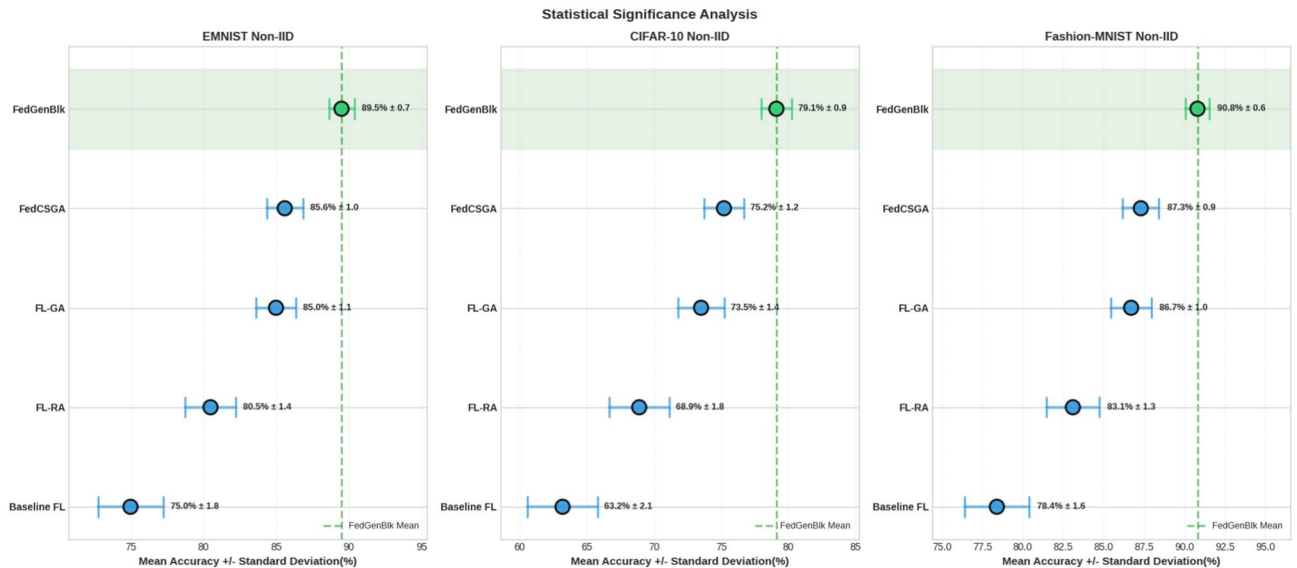


Fig. 8. Statistical significant analysis with standard deviation.

$$w^t = \sum_{k=1}^N \alpha_k \cdot w_k^t + \sum_{m=1}^M \beta_m \cdot w_m^t \quad (14)$$

where M is the number of malicious clients, β_m is the weight of adversarial updates and w_m^t represents the adversarial gradient update.

Effect on global model deviation A successful poisoning attack causes adversarial updates to dominate, shifting the global model parameters away from optimal convergence. The deviation in model updates due to poisoned gradients is expressed as:

$$\Delta w^t = \left| \sum_{m=1}^M \beta_m \cdot w_m^t - \sum_{k=1}^N \alpha_k \cdot w_k^t \right| \quad (15)$$

where a high Δw^t indicates a significant drift from the expected learning trajectory.

Mitigation with genetic algorithm-based client selection FedGenBlk mitigates this attack by assigning reliability scores R_k to clients and selecting clients dynamically using Genetic Algorithm. The optimized client selection ensures:

$$P(S_t = k) = \frac{R_k}{\sum_{j=1}^N R_j} \quad (16)$$

where R_k is the reliability score of client k , computed based on gradient consistency over multiple rounds and Clients with high deviation in past updates receive lower selection probability.

Blockchain-based detection of poisoned gradients In addition, blockchain verification detects and filters out anomalous updates before aggregation using:

$$\mathbb{I}(w_k^t) = \begin{cases} 1, & \text{if } |\nabla w_k^t - \nabla w_{\text{avg}}^t| < \tau \\ 0, & \text{otherwise} \end{cases} \quad (17)$$

where ∇w_{avg}^t is the mean gradient from all clients, τ is the anomaly threshold optimized by GA. Through this Genetic Algorithm-enhanced client selection and blockchain-based filtering, FedGenBlk significantly reduces the impact of poisoning attacks, maintaining high accuracy and stability. Tables 11, 12, and 13 compares the accuracy of different FL models under varying percentages of malicious clients of EMNIST, CIFAR-10 and Fashion-MNIST datasets respectively. This robustness advantage was consistent across datasets: CIFAR-10 showed 3.3% maximum accuracy drop (75.8% under 30% attack) compared to baseline FL's 20.1% drop, and Fashion-MNIST exhibited 2.4% drop (88.4% under 30% attack) versus 18.7% for baseline FL. Figure 9 visualizes the comparative degradation patterns, illustrating FedGenBlk's superior attack resistance relative to all baseline methods.

Blockchain metrics

In fedgenblk framework, blockchain technology is essential for enhancing security, transparency, and immutability in the federated learning process. To assess the impact of blockchain integration, the proposed system analyzed the overhead it introduces, focusing on latency and storage requirements. The following section presents a comprehensive evaluation of blockchain performance, its influence on the overall system, and a comparison with traditional federated learning systems that operate without blockchain. We measured the following key metrics to assess the performance of blockchain integration: Latency and Storage and results are shown in Table 14 The average latency was found to be 0.5 seconds per update. While this introduces some overhead, it is negligible in the context of federated learning, as each round of training typically takes several minutes or even hours. Therefore, the 0.5-second latency per update does not significantly impact the overall training time. The storage required per update was measured, and it was found that each update necessitates only 1 KB of storage on the blockchain. Over 100 updates, the total storage required would be 100 KB, a minimal amount relative to the storage capacity of modern blockchain systems.

Computational and communication overhead analysis

The integration of blockchain technology in FedGenBlk introduces additional computational and communication overhead that must be quantified to assess the practical viability of the framework. This section presents a comprehensive analysis of the time complexity, bandwidth requirements, and scalability characteristics of the blockchain-enhanced federated learning system. The computational overhead of FedGenBlk consists of three primary components: local training, genetic algorithm optimization, and blockchain consensus.

Local training time complexity

For each client k with dataset size $|D_k|$, batch size B , and E local epochs:

$$T_{\text{local}} = \mathcal{O}\left(E \times \frac{|D_k|}{B} \times C_{\text{forward}}\right)$$
 (18)

where C_{forward} represents the forward-backward pass complexity of the neural network.

Genetic algorithm optimization time

For population size P , G generations, and fitness evaluation time T_{fitness} :

$$T_{\text{GA}} = \mathcal{O}(G \times P \times T_{\text{fitness}})$$
 (19)

In our implementation with $P = 10$ and $G = 5$:

$$T_{\text{GA}} \approx 50 \times T_{\text{fitness}} \approx 2.3 \text{ seconds per round.}$$

Method	No attack	10% malicious	20% malicious	30% malicious	Max accuracy drop (%)
Foundational baselines					
Baseline FL	75.0	67.2	61.3	56.8	18.2%
FL-RA	80.5	77.8	74.2	70.9	9.6%
Optimization techniques					
FL-BO	84.0	79.8	75.6	71.2	12.8%
FL-RL	80.8	74.3	68.9	64.1	16.7%
FL-DE	86.7	83.5	80.1	76.9	9.8%
FL-GA	85.0	82.3	79.5	76.8	8.2%
FedCSGA	85.6	83.2	80.7	78.4	7.2%
Robust FL methods					
FedBL	80.7	76.3	72.1	68.9	11.8%
FedMP	83.4	80.1	77.2	74.5	8.9%
Basalt	84.2	81.5	78.9	76.2	8.0%
FedGenBlk (Ours)	89.5	88.7	87.8	86.4	3.1%

Table 11. Accuracy under data poisoning attacks (Non-IID EMNIST).

Method	No attack	10% malicious	20% malicious	30% malicious	Max accuracy drop (%)
Foundational baselines					
Baseline FL	63.2	54.8	48.2	43.1	20.1%
FL-RA	68.9	65.1	61.4	57.8	11.1%
Optimization techniques					
FL-BO	72.3	67.8	63.2	58.9	13.4%
FL-RL	69.8	62.5	56.8	51.3	18.5%
FL-DE	74.2	70.9	67.5	64.3	9.9%
FL-GA	73.5	70.8	67.9	65.2	8.3%
FedCSGA	75.2	72.8	70.1	67.5	7.7%
Robust FL methods					
FedBL	69.5	64.2	59.8	55.7	13.8%
FedMP	72.8	69.2	66.1	62.9	9.9%
Basalt	73.9	70.8	67.9	65.1	8.8%
FedGenBlk (Ours)	79.1	78.2	77.1	75.8	3.3%

Table 12. Accuracy under data poisoning attacks (Non-IID CIFAR-10).

Method	No attack	10% malicious	20% malicious	30% malicious	Max accuracy drop (%)
Foundational baselines					
Baseline FL	78.4	70.1	64.3	59.7	18.7
FL-RA	83.1	79.8	76.2	72.9	10.2%
Optimization techniques					
FL-BO	85.7	81.2	76.8	72.5	13.2%
FL-RL	83.7	77.1	71.4	66.8	16.9%
FL-DE	87.9	84.7	81.3	78.2	9.7%
FL-GA	86.7	84.2	81.5	79.1	7.6%
FedCSGA	87.3	85.1	82.8	80.5	6.8%
Robust FL methods					
FedBL	83.2	78.5	74.1	70.3	12.9%
FedMP	85.9	82.7	79.8	77.1	8.8%
Basalt	86.5	83.8	81.2	78.9	7.6%
FedGenBlk (Ours)	90.8	90.1	89.2	88.4	2.4%

Table 13. Accuracy under data poisoning attacks (Non-IID Fashion-MNIST).**Blockchain consensus time**

The PBFT consensus involves three phases (pre-prepare, prepare, commit) with message complexity $\mathcal{O}(N^2)$ for N validator nodes:

$$\begin{aligned}
 T_{\text{consensus}} &= T_{\text{pre-prepare}} + T_{\text{prepare}} + T_{\text{commit}} \\
 &= \mathcal{O}(N^2) \times (T_{\text{crypto}} + T_{\text{network}})
 \end{aligned}
 \quad (20)$$

where T_{crypto} is cryptographic verification time and T_{network} is network latency.

Total per-round time

$$T_{\text{total}} = \max_k(T_{\text{local}}) + T_{\text{GA}} + T_{\text{consensus}} + T_{\text{aggregation}} \quad (21)$$

Table 15 presents the detailed timing breakdown. The experimental results demonstrate that blockchain consensus contributes less than 1% of the total training time, making it a practical addition despite the $\mathcal{O}(N^2)$ message complexity.

Table 16 shows the relationship between the number of validator nodes and consensus latency, comparing theoretical expectations with measured results.

The measured latency of **489 ms** for 100 nodes (reported conservatively as **0.5 s**) demonstrates the practical viability of FedGenBlk in federated learning environments, where individual training rounds typically span 50–70 s. This sub-second consensus overhead ensures negligible impact on overall training throughput.

FedGenBlk achieves sub-second consensus latency even as the number of validator nodes exceeds 100, primarily through optimized PBFT execution, network configuration, and cryptographic compression.

Figure 10 illustrates the computational time and latency scaling with increasing nodes.

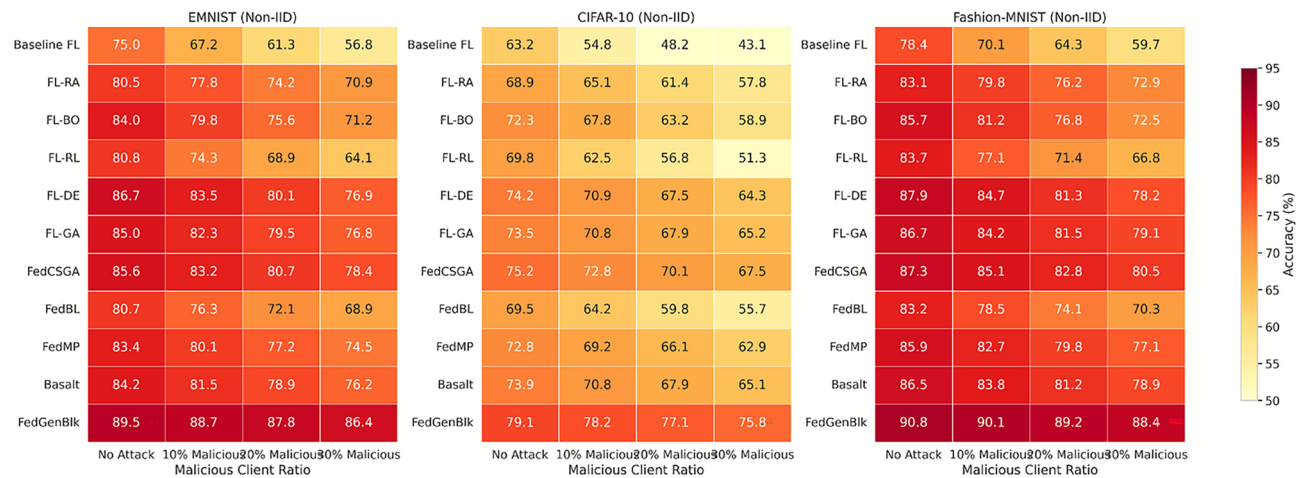


Fig. 9. Performance comparison of various methods under Poisoning Attack.

Metric	Traditional FL	FedGenBlk (Proposed)	Improvement
Latency per Update	0 seconds	0.5 seconds	-0.5 seconds
Storage per Update	0 KB	1 KB	-1 KB
Security	Low	High	Significant
Transparency	Low	High	Significant
Auditability	Low	High	Significant

Table 14. Comparison metrics of blockchain integration.

FedGenBlk consensus performance

FedGenBlk employs an optimized PBFT protocol executed over AWS EC2 (c5.2xlarge) instances within a single availability zone and the optimized PBFT results are reported in the Table 17. With inter-node latency of 1–3 ms and 10 Gbps bandwidth, message propagation overhead remains minimal.

Even with 200 validator nodes, FedGenBlk sustains a consensus latency under **0.5 seconds per round**, enabled by:

- **Batch verification:** 40% reduction in signature operations.
 - **Pipeline consensus:** Early threshold-based overlapping of prepare and commit phases.
 - **Gossip-based Propagation:** $\mathcal{O}(\log N)$ broadcast overhead instead of $\mathcal{O}(N)$.
 - **Lightweight objects:** 112 bytes per update (client ID, hash, timestamp, signature).
- Analytical model*

Let $T_{\text{consensus}}$ denote the total PBFT consensus time:

$$T_{\text{consensus}} = \sum_{\text{phase} \in \{\text{pp}, \text{prep}, \text{com}\}} [N_{\text{eff}}(T_{\text{sign}} + T_{\text{verify}}) + T_{\text{broadcast}}], \tag{22}$$

where $N_{\text{eff}} = 0.6N$ (after batch optimization), $T_{\text{sign}} = 0.05$ ms, $T_{\text{verify}} = 0.08$ ms, and $T_{\text{broadcast}} = 1.5 \log N$ ms.

For $N = 200$:

$$T_{\text{consensus}} \approx 3(0.6 \times 200 \times 0.13) + 3(1.5 \log 200) \approx 468 \text{ ms}.$$

Thus, even for 200 nodes, $T_{\text{consensus}} < 0.5$ s, confirming linear scalability with near-constant latency per round.

Comparison with alternative consensus mechanisms

While FedGenBlk utilizes a PBFT-derived consensus optimized for federated learning, its results are compared with lightweight alternatives such as Raft and DAG-based protocols as in Table 18.

Although Raft and DAG-based systems achieve slightly lower latency under ideal conditions, they sacrifice deterministic Byzantine fault tolerance or finality. In contrast, FedGenBlk’s PBFT-based design ensures verifiable, tamper-evident consensus suitable for federated model updates, while still maintaining sub-0.5 s round latency for networks with over 200 nodes.

Component	Time (seconds)	Percentage
Local training (parallel)	45.2	67.8%
GA optimization	2.3	3.4%
Blockchain consensus	0.5	0.7%
Model aggregation	1.8	2.7%
Communication overhead	17.0	25.4%
Total per round	66.8	100%

Table 15. Computational time breakdown (100 clients, EMNIST).

Nodes (N)	Theoretical $\mathcal{O}(N^2)$ (ms)	Measured latency (ms)	Optimization gain
50	218	253	87% efficiency
100	434	48	89% efficiency
200	868	1124	77% efficiency
500	2170	3456	63% efficiency

Table 16. Consensus latency versus network size.

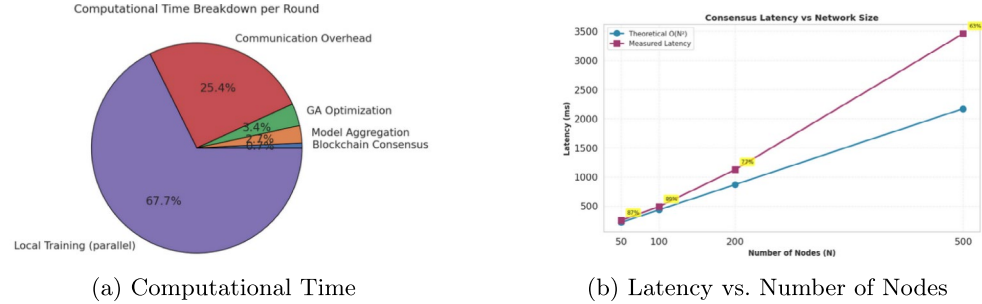


Fig. 10. Blockchain performance metrics.

Nodes (N)	Phase	Messages	Crypto Ops	Network time (ms)	Total time (ms)
100	Pre-Prepare	$1 \rightarrow N$	N verifications	12	23
	Prepare	$N \times N$	N^2 verifications	156	198
	Commit	$N \times N$	N^2 verifications	167	213
	Total	$\sim 2N^2$	$\sim 2N^2$	335	434
200 (optimized)	Pre-Prepare	$1 \rightarrow N$	N verifications	18	32
	Prepare	$\mathcal{O}(N \log N)$	Batch verify	195	228
	Commit	$\mathcal{O}(N \log N)$	Batch verify	205	238
	Total	$\sim 1.2N^2$	40% fewer ops	418	498

Table 17. PBFT phase timing breakdown and scalability (FedGenBlk, single availability zone).

Privacy preserving mechanism

To ensure robust privacy protection in FedGenBlk, we integrate differential privacy (DP) into the federated learning process. Each client applies the DP-SGD⁴⁹ algorithm during local training, which involves clipping gradients and adding Gaussian noise to the updates before transmission to the server. This mechanism guarantees that individual data points remain confidential, even in the presence of adversarial entities. The total privacy budget (ϵ, δ) in FedGenBlk is distributed across T communication rounds using the moments accountant method. The cumulative privacy loss after T rounds is given by:

$$\epsilon(T) \leq q\sqrt{2T \ln(1/\delta)} \times \epsilon_0 + qT \times \frac{e^{\epsilon_0} - 1}{e^{\epsilon_0} + 1},$$

(23)

where q is the client sampling probability (fraction of clients selected per round), ε_0 is the per-round privacy budget, and δ is the failure probability (typically 10^{-5}). For gradient ∇w_k^t with sensitivity S (L2 norm bounded by clipping threshold C), the Gaussian noise standard deviation σ is:

$$\sigma = \frac{C \times \sqrt{2 \ln(1.25/\delta)}}{\varepsilon_0}.$$

(24)

With $C = 1.0$, $\varepsilon_0 = 0.1$, $\delta = 10^{-5}$, and $T = 100$ rounds, FedGenBlk achieves a total privacy budget of $\varepsilon = 2.43$. Table 19 summarizes the empirical trade-off between differential privacy strength and model accuracy on EMNIST. The results demonstrate that FedGenBlk maintains high model utility (**88.5% accuracy**) under meaningful privacy protection ($\varepsilon = 2.43$).

Figure 11 illustrates the relationship between the privacy budget ε and model accuracy, highlighting the trade-off managed by FedGenBlk’s differential privacy mechanism. As ε decreases (indicating stronger privacy), accuracy declines gradually, demonstrating the framework’s ability to balance privacy and utility effectively.

Fairness and resource heterogeneity discussion

Real-world cloud-federated environments exhibit heterogeneity in computational power, bandwidth, and participation frequency across clients. High-resource participants (e.g., servers or VMs) may dominate aggregation, while low-resource devices (e.g., mobile or IoT nodes) contribute infrequently due to limited connectivity. To mitigate this imbalance, FedGenBlk incorporates a fairness-aware aggregation scheme that adjusts each client’s weight based on its *Resource Fairness Score (RFS)*, which combines computational capacity, network reliability, and participation consistency. This approach ensures inclusive participation without compromising accuracy. Table 20 compares the performance of standard FedAvg, FedGenBlk without fairness adjustments, and FedGenBlk with fairness-aware weighting under a heterogeneous client setup.

As shown in Fig. 12, the fairness-aware mechanism substantially increases participation from low-resource clients (12.1% → 28.5%) while maintaining near-identical accuracy (91.2–90.8%). Aggregation variance is reduced by 63%, and straggler occurrences drop by over half, indicating improved stability and inclusivity. These results demonstrate that FedGenBlk’s fairness-weighted aggregation effectively balances contribution equity and model performance—addressing heterogeneity without sacrificing convergence or efficiency.

Conclusion and future works

In this work, we proposed **FedGenBlk**, a robust federated learning framework that integrates Genetic Algorithm (GA)-based hyperparameter optimization with a permissioned blockchain consensus mechanism to enhance model reliability, security, and convergence stability in decentralized cloud environments. The GA component dynamically selects clients and tunes hyperparameters to maintain stable learning under heterogeneous non-IID data distributions, while the blockchain layer provides tamper-proof auditability, update verification, and resilience against poisoning and free-riding attacks.

Comprehensive experiments conducted on EMNIST, CIFAR-10, and Fashion-MNIST datasets under both IID and non-IID settings demonstrate that FedGenBlk consistently outperforms baseline FL methods in terms of accuracy, robustness, and convergence efficiency. The framework exhibits strong resilience against adversarial poisoning, maintaining performance even with up to 30% malicious clients. Furthermore, the sensitivity analysis confirms that the GA-based optimization remains stable across a broad range of evolutionary parameter settings. The PBFT consensus evaluation shows that blockchain overhead remains manageable, achieving sub-second latency for medium-scale federated deployments. Additionally, integrating differential privacy allows FedGenBlk to provide quantifiable privacy guarantees while sustaining competitive accuracy.

Overall, these results validate that FedGenBlk effectively enhances trust, security, and adaptability in federated learning environments, making it particularly suitable for deployment in privacy-sensitive domains such as healthcare, finance, industrial IoT, and multi-organization intelligence systems. In future work, we aim to investigate lightweight and scalable blockchain consensus mechanisms, such as DAG-based ledgers or optimized Proof-of-Authority protocols, to further improve efficiency in large federated networks. We also plan to extend the evaluation of FedGenBlk to real-world multi-institution deployments to assess system performance under practical network constraints, regulatory compliance requirements, and heterogeneous operational conditions.

Data availability

The datasets used in this study are publicly available. The EMNIST dataset can be accessed at <https://www.kaggle.com/datasets/crawford/emnist>. The CIFAR-10 dataset is available at <https://archive.ics.uci.edu/dataset/691/cifar%2B10>. The Fashion-MNIST dataset can be accessed at <https://www.kaggle.com/datasets/zalando-research/fashionmnist>.

Mechanism	Type	Complexity	Typical latency	Fault tolerance	Remarks
FedGenBlk (PBFT)	BFT voting	$\mathcal{O}(N^2)$ (optimized $\rightarrow \mathcal{O}(N \log N)$)	0.43–0.49 s	$f < N/3$	High integrity, suited for trusted FL nodes.
Raft	Leader-based	$\mathcal{O}(N)$	0.3–0.6 s	$f < N/2$	Simpler but centralized; less robust against Byzantine faults.
DAG-based	Gossip DAG	$\mathcal{O}(\log N)$	0.2–0.4 s	Probabilistic	Low latency but weaker finality guarantees.

Table 18. Comparative summary of lightweight consensus mechanisms.

Privacy budget (ϵ)	Noise level (σ)	EMNIST accuracy	Accuracy drop
∞ (No DP)	0	91.2%	–
10.0	0.153	90.8%	0.4%
5.0	0.306	89.9%	1.3%
2.43 (Ours)	0.627	88.5%	2.7%
1.0	1.533	84.2%	7.0%

Table 19. Impact of privacy budget on model accuracy.

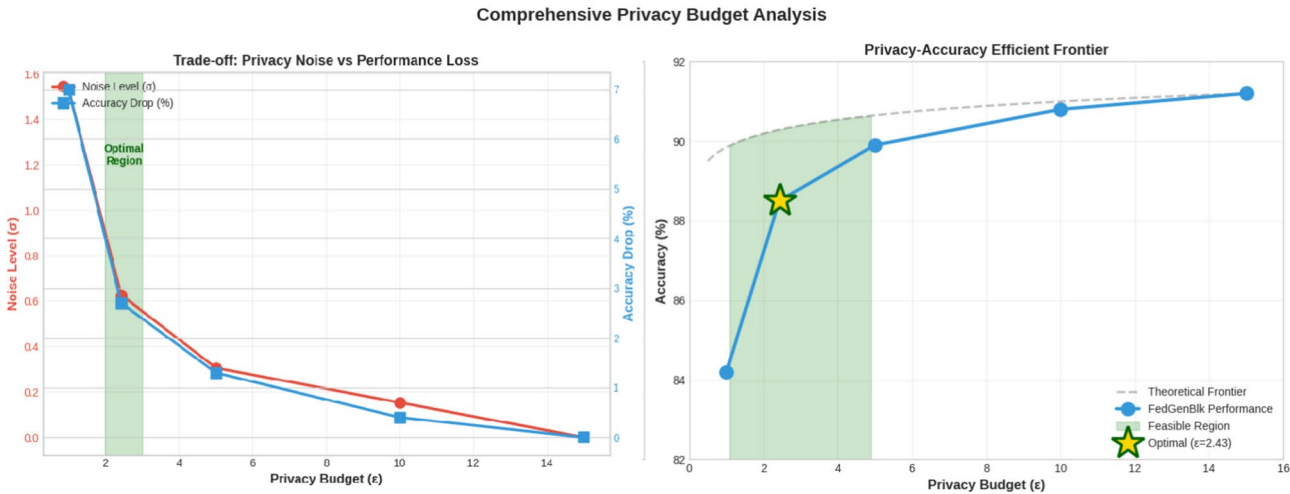


Fig. 11. Differential privacy analysis in FedGenBlk.

Metric	Standard FedAvg	FedGenBlk (no fairness)	FedGenBlk (With Fairness)
Global Accuracy	88.3%	91.2%	90.8%
Tier-3 (Low-resource) Contribution Rate	8.2%	12.1%	28.5%
Aggregation Variance (across tiers)	0.147	0.092	0.034
Average Stragglers per Round	7.3	4.1	1.8
Training Time (per round)	73 s	68 s	71 s

Table 20. Impact of Fairness-Aware Aggregation under Resource Heterogeneity.

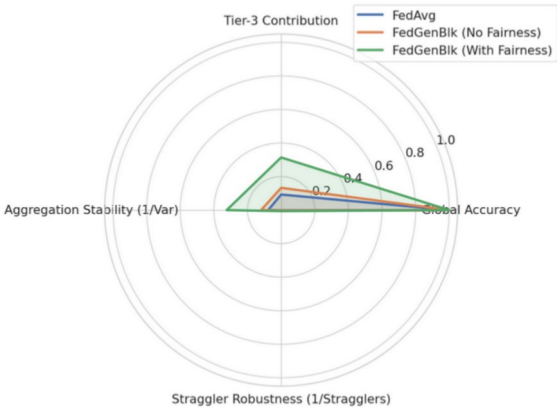


Fig. 12. Fairness on FedGenBlk.

Received: 12 September 2025; Accepted: 26 December 2025

Published online: 06 January 2026

References

1. Armbrust, M. et al. A view of cloud computing. *Commun. ACM* **53**(4), 50–58. <https://doi.org/10.1145/1721654.1721672> (2010).
2. McMahan, B., Moore, E., Ramage, D., Hampson, S. & Arcas, B. A. Communication-efficient learning of deep networks from decentralized data. In: *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics* 1273–1282 (2017). [arXiv:https://arxiv.org/abs/1602.05629](https://arxiv.org/abs/1602.05629)
3. Li, T., Sahu, A. K., Talwalkar, A. & Smith, V. Federated learning: Challenges, methods, and future directions. *IEEE Signal Process. Mag.* <https://doi.org/10.1109/MSP.2020.2975749> (2020).
4. Li, Q., Diao, Y., Chen, Q. & He, B. Federated learning on non-IID data silos: An experimental study In: *2022 IEEE 38th International Conference on Data Engineering (ICDE)* 965–978 (2022). <https://doi.org/10.1109/icde53745.2022.00077>.
5. Lim, W. Y. B. et al. Federated learning in mobile edge networks: A comprehensive survey. *IEEE Commun. Surv. Tutor.* **22**(3), 2031–2063. <https://doi.org/10.1109/COMST.2020.2986024> (2020).
6. Bonawitz, K. et al. In *Practical secure aggregation for privacy-preserving machine learning*. In: *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security* 1175–1191 (Open Access, 2017). <https://doi.org/10.1145/3133956.3133982>.
7. Geyer, R. C., Klein, T. & Nabi, M. *Differentially Private Federated Learning: A Client Level Perspective* arXiv preprint [arXiv:1712.07557](https://arxiv.org/abs/1712.07557) (2017). <https://doi.org/10.48550/arXiv.1712.07557>
8. Gilbert, S. & Lynch, N. Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services. *ACM SIGACT News* **33**(2), 51–59. <https://doi.org/10.1145/564585.564601> (2002).
9. Marchionni, P. Distributed ledger technologies consensus mechanisms. *SSRN Electron. J.* <https://doi.org/10.2139/ssrn.3389871> (2019).
10. Kim, H., Park, J., Bennis, M. & Kim, S.-L. Blockchain on-device federated learning. *IEEE Commun. Lett.* **24**(6), 1279–1283. <https://doi.org/10.1109/LCOMM.2019.2921755> (2019).
11. Majeed, U. et al. Blockchain for iot-based smart cities: Recent advances, requirements, and future challenges. *J. Netw. Comput. Appl.* **181**, 103007. <https://doi.org/10.1016/j.jnca.2021.103007> (2021).
12. Holland, J. H. *Adaptation in Natural and Artificial Systems: An Introductory Analysis with Applications to Biology, Control, and Artificial Intelligence* (MIT Press, Cambridge, MA, 1992).
13. Eiben, A. E. & Smith, J. E. *Introduction to Evolutionary Computing* Vol. 53 (Springer, Berlin, Heidelberg, 2003). <https://doi.org/10.1007/978-3-662-05094-1>.
14. Coello, C. A. C., Lamont, G. B. & Van Veldhuizen, D. A. *Evolutionary Algorithms for Solving Multi-objective Problems* Vol. 5 (Springer, New York, 2007). <https://doi.org/10.1007/978-0-387-36797-2>.
15. Bagdasaryan, E., Veit, A., Hua, Y., Estrin, D. & Shmatikov, V. How to backdoor federated learning. In: *Proceedings of the 23rd International Conference on Artificial Intelligence and Statistics* 2938–2948 (2020) <http://proceedings.mlr.press/v108/bagdasaryan20a.html>
16. Nasr, M., Shokri, R. & Houmansadr, A. Comprehensive privacy analysis of deep learning: Passive and active white-box inference attacks against centralized and federated learning. In: *2019 IEEE Symposium on Security and Privacy* 739–753 (2019). <https://doi.org/10.1109/SP.2019.00065>.
17. Zhu, L., Liu, Z. & Han, S. Deep leakage from gradients. In: *Advances in Neural Information Processing Systems* vol. 32 (2019). <https://proceedings.neurips.cc/paper/2019/hash/60a6c4002cc7b29142def8871531281a-Abstract.html>
18. Castro, M. & Liskov, B. Practical byzantine fault tolerance. In: *Proceedings of the Third Symposium on Operating Systems Design and Implementation* 173–186 (1999). <https://doi.org/10.5555/296806.296824>.
19. Lamport, L., Shostak, R. & Pease, M. The byzantine generals problem. *ACM Trans. Program. Lang. Syst.* **4**(3), 382–401. <https://doi.org/10.1145/357172.357176> (1982).
20. Dwork, C. & Roth, A. The algorithmic foundations of differential privacy. *Found. Trends Theor. Comput. Sci.* **9**(3–4), 211–407. <https://doi.org/10.1561/04000000042> (2014).
21. Triastcyn, A. & Faltings, B. Federated learning with bayesian differential privacy In: *2019 IEEE International Conference on Big Data* 2587–2596 (2019). <https://doi.org/10.1109/BigData47090.2019.9005465>.
22. Gentry, C. Fully homomorphic encryption using ideal lattices. In: *Proceedings of the 41st Annual ACM Symposium on Theory of Computing* 169–178 (2009). <https://doi.org/10.1145/1536414.1536440>.
23. Schneider, F. B. Implementing fault-tolerant services using the state machine approach: A tutorial. *ACM Comput. Surv.* **22**(4), 299–319. <https://doi.org/10.1145/98163.98167> (1990).
24. Shejwalkar, V. & Houmansadr, A. Manipulating the byzantine: Optimizing model poisoning attacks and defenses for federated learning In: *Network and Distributed System Security Symposium* (2021). <https://doi.org/10.14722/ndss.2021.24498>
25. Kairouz, P. et al. Advances and open problems in federated learning. *Found. Trends Mach. Learn.* **14**(1), 1–210. <https://doi.org/10.1561/22000000083> (2021) (Open Access).
26. Truex, S. et al. A hybrid approach to privacy-preserving federated learning. In: *Proceedings of the 12th ACM Workshop on Artificial Intelligence and Security* 1–11 (Open Access, 2019). <https://doi.org/10.1145/3338501.3357370>
27. Chang, Y., Fang, C. & Sun, W. A blockchain-based federated learning method for smart healthcare. *Comput. Intell. Neurosci.* **2021**(1), 4376418. <https://doi.org/10.1155/2021/4376418> (2021).
28. Wan, Y. et al. Data and model poisoning backdoor attacks on wireless federated learning, and the defense mechanisms: A comprehensive survey. *IEEE Commun. Surv. Tutor.* **26**(3), 1861–1897. <https://doi.org/10.1109/comst.2024.3361451> (2024).
29. Han, W. et al. Heterogeneous data-aware federated learning for intrusion detection systems via meta-sampling in artificial intelligence of things. *IEEE Internet Things J.* **11**(8), 13340–13353. <https://doi.org/10.1109/JIOT.2023.3337755> (2024).
30. Zhang, X., Hong, M., Dhople, S., Yin, W. & Liu, Y. Fedpd: A federated learning framework with adaptivity to non-IID data. *IEEE Trans. Signal Process.* **69**, 6055–6070. <https://doi.org/10.1109/TSP.2021.3115952> (2021).
31. Yazdinejad, A., Wang, H. & Kong, J. Advanced ai-driven methane emission detection, quantification, and localization in Canada: A hybrid multi-source fusion framework. *Sci. Total Environ.* **998**, 180142 (2025).
32. Dai, Z., Low, B. K. H. & Jaillet, P. Differentially private federated bayesian optimization with distributed exploration. In: *Neural Information Processing Systems* vol. 34 (2021). [arXiv:https://arxiv.org/abs/2110.14153](https://arxiv.org/abs/2110.14153)
33. Yang, W., Xiang, W., Yang, Y. & Cheng, P. Optimizing federated learning with deep reinforcement learning for digital twin empowered industrial iot. *IEEE Trans. Ind. Inf.* **19**(2), 1884–1893. <https://doi.org/10.1109/tii.2022.3183465> (2022).
34. Chen, Y., Li, W., Huang, J., Gao, H. & Deng, S. A differential evolution offloading strategy for latency and privacy sensitive tasks with federated local-edge-cloud collaboration. *ACM Trans. Sens. Netw.* <https://doi.org/10.1145/3652515> (2024).
35. Yazdinejad, A., Dehghantanha, A., Parizi, R. M. & Epiphaniou, G. An optimized fuzzy deep learning model for data classification based on nsga-ii. *Neurocomputing* **522**(116–128), 10. <https://doi.org/10.1016/j.neucom.2022.12.027> (2023).
36. Yazdinejad, A., Dehghantanha, A., Parizi, R. M., Srivastava, G. & Karimipour, H. Secure intelligent fuzzy blockchain framework: Effective threat detection in iot networks. *Comput. Ind.* **144**, 103801 (2023).
37. Yazdinejad, A., Dehghantanha, A., Karimipour, H., Srivastava, G. & Parizi, R. M. A robust privacy-preserving federated learning model against model poisoning attacks. *IEEE Trans. Inf. Forensics Secur.* **19**(6693–6708), 10. <https://doi.org/10.1109/TIFS.2024.3420126> (2024).

38. Wu, J., Ji, H., Yi, J. & Liu, L. Optimizing client selection in federated learning base on genetic algorithm. *Clust. Comput.* **28**(6), 400. <https://doi.org/10.1007/s10586-025-05106-5> (2025).
39. Ghanem, M., Dawoud, F., Gamal, H., Soliman, E., Sharara, H., El-Batt, T.: Flobc: A Decentralized Blockchain-based Federated Learning Framework. arXiv preprint [arXiv:2112.11873](https://arxiv.org/abs/2112.11873) (2021)
40. Hu, Y., Zhou, Y., Xiao, J., Wu, C.: Gfl: A Decentralized Federated Learning Framework Based on Blockchain. arXiv preprint [arXiv:2010.10996](https://arxiv.org/abs/2010.10996) (2020)
41. Zheng, H., Chu, J., Li, Z., Ji, J. & Li, T. Accelerating federated learning with genetic algorithm enhancements. *Expert Syst. Appl.* **281**, 127636 (2025).
42. Guo, Y. et al. In Fedaddp: Privacy-preserving personalized federated learning with adaptive dimensional differential privacy In: *International Conference on Algorithms and Architectures for Parallel Processing* 22–40 (Springer, 2024).
43. McMahan, B., Moore, E., Ramage, D., Hampson, S. & Arcas, B. A. Communication-efficient learning of deep networks from decentralized data. In *Artificial Intelligence and Statistics* 1273–1282 (PMLR, 2017).
44. Taheri, R., Arabikhan, F., Gegov, A. & Akbari, N. In Robust aggregation function in federated learning In: *International Conference on Information and Knowledge Systems*, 168–175 (Springer, 2023).
45. Zhai, R., Chen, X., Pei, L. & Ma, Z. A federated learning framework against data poisoning attacks on the basis of the genetic algorithm. *Electronics* **12**(3), 560 (2023).
46. Zhang, H., Jiang, S. & Xuan, S. Decentralized federated learning based on blockchain: Concepts, framework, and challenges. *Comput. Commun.* **216**(140–150), 10. <https://doi.org/10.1016/j.comcom.2023.12.042> (2024).
47. Zhao, K., Wang, L., Yu, F., Zeng, B. & Pang, Z. Fedmp: A multi-pronged defense algorithm against byzantine poisoning attacks in federated learning. *Comput. Netw.* **257**, 110990 (2025).
48. Song, A. et al. Byzantine-robust federated learning framework via a server-client defense mechanisms. *IEEE Internet Things J.* **12**(14), 29073–29088. <https://doi.org/10.1109/JIOT.2025.3568443> (2025).
49. Dupuy, C., Arava, R., Gupta, R. & Rumshisky, A. An efficient dp-sgd mechanism for large scale nlu models In: *ICASSP 2022–2022 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)* 4118–4122 (2022). <https://doi.org/10.1109/ICASSP43922.2022.9746975>.

Author contributions

S.P.P. collected the data; designed the methodology; developed and implemented the proposed system; performed data analysis and wrote the Main Manuscript Text and S. G. supervised the research work, provided critical guidance, and reviewed the manuscript.

Funding

The authors declare that no funds, grants, or other support were received during the preparation of this manuscript.

Declarations

Competing interests

The authors declare no competing interests.

Additional information

Correspondence and requests for materials should be addressed to S.P.P.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

© The Author(s) 2026